Università degli Studi di Napoli

“Federico II”

Scuola Politecnica delle Scienze di Base
Dipartimento di Ingegneria Elettrica e Tecnologie dell’Informazione
D.I.E.T.I

Tesi di Laurea
in
Ingegneria Biomedica

Pipeline di segmentazione normalizzata per la classificazione automatica dello stato di attivazione di Linfociti T

(Normalized segmentation pipeline for automatic T-cell activation state classification)

Relatore

Ch.mo Prof. David Dannhauser

Candidato

Giovanni Ruocco
Matr.: P46/432

Anno Accademico 2024/2025

A chi penso quando guardo il cielo

Abstract

This work arises from the need to make cellular analysis more efficient, starting from the study of morphology up to parameters related to microscopy acquisition conditions, such as image brightness, intensity, and contrast. The project focuses on T lymphocytes at different stages of their life cycle and exploits the Cellpose-SAM segmentation model to automatically segment the image dataset and extract, for each cell, a broad set of quantitative parameters. The extracted features are analyzed using statistical methods and both linear and nonlinear machine learning models, with a dual objective: on the one hand, filtering high-quality cells from artefactual or unusable ones, and on the other hand assigning each cell to specific functional states (naive, active, exhausted). Particular attention is devoted to the distinction between active and exhausted T lymphocytes, a topic of great clinical interest because the presence of exhausted populations can reflect the immune system's response to chronic infections or tumors and represents a potential target for advanced diagnostic and therapeutic strategies. From an engineering perspective, the pipeline implemented in Python and optimized for GPU execution constitutes a reusable workflow for quantitative bioimaging applications, which can be extended to other cell types and imaging modalities.

Sommario

Questo lavoro nasce dall'esigenza di rendere più efficienti le analisi cellulari, partendo dallo studio della morfologia fino ai parametri legati alle caratteristiche di acquisizione microscopica, come luminosità, intensità e contrasto dell'immagine. In particolare, il progetto si concentra sui linfociti T in diverse fasi della loro vita: mediante il modello di segmentazione Cellpose-SAM è stato possibile segmentare automaticamente il dataset di immagini e ricavare un ampio insieme di parametri quantitativi per ogni cellula. Tali feature sono state poi analizzate tramite metodi statistici e modelli di machine learning lineari e non lineari, con l'obiettivo di filtrare le cellule di buona qualità da quelle artefatte o inutilizzabili e, successivamente, di associare ogni cellula a specifici stati funzionali (vergine, attiva, esausta). L'attenzione è rivolta in particolare alla distinzione tra linfociti T attivi ed esausti, un problema di grande interesse clinico in quanto la presenza di popolazioni esauste può riflettere la risposta del sistema immunitario a infezioni croniche o tumori e rappresenta un potenziale bersaglio per strategie diagnostiche e terapeutiche avanzate.

Tavola delle figure e tabelle

Figura 1.1 – Schema di microscopia brightfield. 3
Figura 2.1 – Esempi di linfociti T nelle tre categorie: Naive, Activated, Exhausted. 12
Figura 2.2 – Esempi di immagini di linfociti nei dataset di test (D1, 1Splus). 13
Figura 2.3 – Logo e interfaccia di Google Colab. 13
Figura 2.4 – Tipologie di GPU offerte da Google Colab. 14
Figura 2.5 – Principali librerie Python installate e utilizzate nella pipeline. 15
Figura 3.1 – Esempio di un'immagine originale del dataset. 35
Figura 3.2 – Esempio di un'immagine equalizzata del dataset. 35
Figura 3.3 – Maschere. 36
Figura 3.4 – Bounding box sulle cellule etichettate. 36
Figura 3.5 – Struttura di salvataggio delle maschere di segmentazione. 36
Figura 3.6 – Cartella di salvataggio dei crops good e visualizzazione di alcuni esempi. 36
Figura 3.7 – Learning curve del modello Logistic Regression per la classificazione good/bad. 38
Figura 3.8 – Learning curve del modello Random Forest per la classificazione good/bad 38
Figura 3.9 – Matrice di confusione per la classificazione Naive/Activated/Exhausted – Random Forest. 41
Figura 3.10 – Matrice di confusione per la classificazione Naive/Activated/Exhausted. (Log_Reg). 41
Figura 3.11 – Matrice di confusione per la classificazione Naive/Activated/Exhausted Ensembled. 41
Figura 3.12 – Mappa PCA delle cellule good nelle tre categorie funzionali. 42
Figura 3.13 – Mappa t-SNE delle cellule good nelle tre categorie funzionali. 42
Figura 3.14 – Mappe PCA per coppie di categorie funzionali. 44
Figura 3.15 – Mappe t SNE per coppie di categorie funzionali. 45
Figura 4.1 – Esempio di immagine del dataset D1. 46
Figura 4.2 – Esempio di immagine del dataset 1Splus. 46
Figura 4.3 – Distribuzione delle classi predette nel dataset 1Splus. 47
Figura 4.4 – Distribuzione delle classi predette nel dataset D1. 47
Figura 4.5 – Numero di cellule classificate nel dataset 1Splus. 47
Figura 4.6 – Numero di cellule classificate nel dataset D1. 47

Tabella 3.1 – Classification report Logistic Regression per la classificazione good/bad. 37
Tabella 3.2 – Classification report Random Forest per la classificazione good/bad. 37
Tabella 3.3 – Feature importance (Log_Reg e Rand_For) per la classificazione good/bad. 39
Tabella 3.4 – Classification report Random Forest per la classificazione Naive/Activated/Exhausted. 40
Tabella 3.5 – Classification report Logistic Regression per la classificazione Naive/Activated/Exhausted. 40
Tabella 3.6 – Top 10 loadings delle prime tre componenti principali (PCA) sulle feature cellulari. 43

Indice

Abstract iv
Sommario vi
Capitolo 1 Introduzione 1
1.1 Contesto biologico: Linfociti T 2
1.2 Brightfield microscopy 3
1.3 Image processing e reti neurali 4
1.3.2 Segmentazione e Tecniche classiche 5
1.3.3 Reti neurali convoluzionali per imaging e bioimaging 6
1.3.4 Modelli per la segmentazione 7
1.4 Modelli di classificazione supervisionata 8
1.4.1 Logistic regression 8
1.4.2 Random Forest 9
1.5 Tecniche di riduzione dimensionale 9
1.5.1 Principal Component Analysis 10
1.5.2 t-Distributed Stochastic Neighbor Embedding 10
Capitolo 2 Metodologia 12
2.1 Dataset 12
2.2 Ambiente di calcolo e librerie 13
2.2.1 Google Colab 13
2.2.2 Python e Librerie utilizzate 14
2.3 Pipeline di Image processing 16
2.3.1 Pre-processing in Python 16
2.3.2 Segmentazione con Cellpose-SAM e post-processing in Python 17
2.4 Estrazione delle feature 19
2.4.1 Feature morfologiche 20
2.4.2 Feature fotometriche 22
2.4.3 Feature combinate 24

2.5 Costruzione dei DataFrame 25
2.6 Modelli di analisi supervisionata delle cellule 26
2.6.1 Etichettatura manuale 26
2.6.2 Applicazione dei modelli di classificazione “Good”/”bad” 28
2.6.3 PCA e t-SNE per l’esplorazione dei cluster di cellule 30
2.6.4 Modelli per la classificazione delle categorie 32
Capitolo 3 Risultati 34
3.1 Descrizione complessiva della pipeline 34
3.2 Risultati di segmentazione 35
3.3 Risultati di classificazione 37
3.3.1 Classificazione good / bad 37
3.3.2 Classificazione Naive vs Activated vs Exhausted 40
3.3.3 Classificazione tra Linfociti T Attivi vs Exhausted 44
Capitolo 4 Conclusioni 46
4.1 Validità del workflow sui linfociti T 46
4.2 Limiti del dataset e della microscopia Brightfield 48
4.3 Possibili estensioni: altri tipi cellulari e modalità di imaging 48
4.4 Contributi e prospettive applicative 49
Bibliografia 51

Capitolo 1 Introduzione

L'analisi quantitativa delle cellule in immagini di microscopia riveste un ruolo fondamentale nell'ingegneria biomedica, perché consente di ottenere misure oggettive sulla morfologia e sullo stato funzionale delle cellule, con ricadute sia nella ricerca di base sia nella pratica clinica. In questo contesto, i linfociti T rappresentano un caso di studio particolarmente rilevante, in quanto possono trovarsi in diversi stati (naive, attivi, esausti) che si riflettono anche in caratteristiche morfologiche e radiometriche osservabili nelle immagini.

La microscopia brightfield, pur permettendo l'osservazione non invasiva di cellule vive, pone sfide significative alla segmentazione automatica a causa del basso contrasto, dei bordi sfumati e della presenza di artefatti ottici, rendendo necessario l'impiego di tecniche di pre-processing e di modelli di segmentazione robusti alle variazioni sperimentali. In questo quadro si colloca l'utilizzo di Cellpose-SAM per ottenere maschere di segmentazione a livello di singola cellula, a partire dalle quali vengono estratte feature morfologiche e fotometriche su cui si basano modelli statistici e di machine learning per la valutazione della qualità della segmentazione e per la classificazione dello stato funzionale dei linfociti T.

L'obiettivo di questo lavoro è progettare, implementare e validare una pipeline che consenta di segmentare automaticamente immagini brightfield di linfociti T, estrarre in modo sistematico parametri quantitativi per ogni cellula e addestrare modelli supervisionati capaci di distinguere cellule "good" da cellule artefatte e di classificare le principali categorie funzionali (Naive, Activated, Exhausted). La pipeline è stata sviluppata in Python su Google Colab, sfruttando librerie di image processing e machine learning e l'accelerazione GPU, così da ottenere un workflow riproducibile, scalabile e riutilizzabile in diversi contesti di bioimaging quantitativo.

1.1 Contesto biologico: Linfociti T

Tra le popolazioni cellulari di maggior interesse in ambito immunologico e oncologico, i linfociti T (o T cells) svolgono un ruolo centrale nel sistema immunitario adattativo, essendo responsabili del riconoscimento e dell'eliminazione di cellule infette o trasformate. Essi rappresentano circa il 60–80% dei linfociti circolanti e derivano da precursori ematopoietici del midollo osseo, completando il loro processo di maturazione nel timo attraverso un rigoroso meccanismo di selezione positiva e negativa, indispensabile per garantire il riconoscimento degli antigeni presentati dal Complesso Maggiore di Istocompatibilità (MHC) e prevenire le reazioni autoimmuni. Una volta maturi, i linfociti T si distinguono principalmente in due sottotipi funzionali: i CD4+ Helper, deputati alla coordinazione della risposta immunitaria tramite la secrezione di citochine, e i CD8+ Citotossici, direttamente coinvolti nell'eliminazione delle cellule infette o anomale. Dal punto di vista funzionale, tali cellule possono trovarsi in diversi stati, passando da una fase naive (non ancora attivata) a una fase attiva (caratterizzata da intensa attività proliferativa ed effettrice) fino a una condizione di esaurimento funzionale (T cell exhaustion), tipica di contesti patologici cronici quali infezioni virali persistenti o microambienti tumorali, in cui la capacità citotossica e proliferativa risulta significativamente ridotta.

Alla luce di tali considerazioni, il presente lavoro di tesi propone lo sviluppo di un codice computazionale basato sul modello Cellpose-SAM (Segment Anything Model), che integra reti neurali convoluzionali con tecniche di segmentazione tradizionali, al fine di segmentare, analizzare e classificare in modo automatico differenti tipologie cellulari in immagini di microscopia. L'applicazione è stata orientata in particolare a dati di microscopia in brightfield, con focus sui linfociti T, con l'obiettivo di discriminare cellule idonee da cellule danneggiate o inutilizzabili per l'analisi e, più specificamente, distinguere linfociti T attivi da linfociti T esausti. Tale approccio mira a fornire un supporto oggettivo e automatizzato all'analisi cellulare in diversi contesti diagnostici e sperimentali, con potenziali estensioni verso ulteriori tipologie cellulari e modalità di imaging.

1.2 Brightfield microscopy

La microscopia in campo chiaro (brightfield microscopy) è una delle tecniche di imaging ottico più comuni e fondamentali in biologia cellulare, grazie alla sua estrema semplicità: il campione viene illuminato dal basso con luce bianca trasmessa, creando un'immagine scura su sfondo luminoso basata sul contrasto naturale dovuto ad assorbimento e dispersione della luce. Rispetto ad altri tipi di microscopie, come la microscopia a fluorescenza, offre il grande vantaggio di osservare cellule vive senza marcatori o modifiche genetiche, preservando il loro stato naturale e fornendo dettagli morfologici ricchi, come nucleo e citoplasma visibili simultaneamente. Tuttavia, queste caratteristiche portano anche sfide importanti per l'analisi automatica e la segmentazione: l'intensità luminosa delle cellule è spesso simile a quella del mezzo di coltura circostante, con basso contrasto tra corpo cellulare e sfondo o tra organelli; i livelli di grigio nel citoplasma non sono uniformi, complicando i confini; artefatti come l'alone (*halo*) (una corona luminosa intorno alla cellula) nascondono i veri margini causando sottostime dell'area; i bordi cellulari risultano frequentemente sfumati, e in colture dense le membrane adiacenti si sovrappongono, rendendo difficile la separazione automatica. Proprio per questo, pipeline avanzate come quella proposta qui (con Cellpose-SAM integrato a pre-elaborazioni come CLAHE e riduzione del rumore) sono essenziali per provare a vincere i limiti della brightfield nell'analisi quantitativa dei linfociti T.

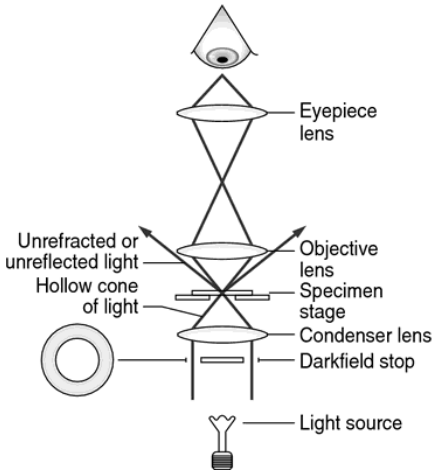


Figura 1.1 Brightfield microscope

1.3 Image processing e reti neurali

L'elaborazione delle immagini (image processing) rappresenta un insieme di tecniche volte all'acquisizione, analisi e trasformazione di immagini digitali, con l'obiettivo di estrarre informazioni o migliorare la qualità visiva dei dati. Ogni immagine digitale può essere modellata come una funzione bidimensionale $f(x, y)$, dove x e y rappresentano le coordinate spaziali e $f(x, y)$ l'intensità luminosa.

Le reti neurali convoluzionali (*Convolutional Neural Network, CNN*) hanno rivoluzionato questo campo, grazie alla capacità di apprendere automaticamente rappresentazioni gerarchiche dei dati visivi attraverso operazioni di convoluzione:

$$(I * K)(x, y) = \sum_m \sum_n I\,(x - m, y - n)\, K(m, n)$$

dove I è l'immagine di input e K il filtro (o *kernel*). Tale meccanismo consente di mimare il funzionamento delle reti neurali umane, rilevando pattern locali, caratteristiche e strutture visive a differenti livelli di astrazione, fornendo una base solida per applicazioni come il riconoscimento facciale, la segmentazione e la classificazione automatica di oggetti. Nel capitolo successivo vedremo come sono stati affrontati questi argomenti dal punto di vista della pipeline.

1.3.1 Pre-processing

La pre-elaborazione di un'immagine consiste nell'applicazione di passaggi volti a ridurre la dimensionalità dei dati in ingresso, stabilizzare la distribuzione delle intensità e produrre un input più omogeneo e numericamente ben condizionato per le reti neurali. In questo lavoro vengono utilizzate tre operazioni principali di pre-elaborazione: conversione in scala di grigi, miglioramento locale del contrasto (tramite CLAHE) e normalizzazione statistica dei pixel.

La conversione in scala di grigi trasforma l'immagine RGB in una funzione di intensità monodimensionale $f(x, y)$, eliminando le componenti di crominanza e preservando la luminanza percepita. Una formulazione tipica è:

$$f(x, y) = 0.299\, R(x, y) + 0.587\, G(x, y) + 0.114\, B(x, y)$$

che tiene conto della diversa sensibilità dell'occhio umano alle tre componenti cromatiche e consente di ridurre da tre a un canale mantenendo l'informazione strutturale rilevante per la segmentazione.

Il filtro CLAHE (*Contrast Limited Adaptive Histogram Equalization*) esegue un'equalizzazione dell'istogramma in modo adattivo su piccole regioni (tile) dell'immagine, limitando però il contrasto per evitare l'amplificazione del rumore. L'istogramma di ciascun tile viene troncato a un valore di soglia (clip limit) e la parte eccedente viene redistribuita sui livelli di intensità, producendo una trasformazione locale che aumenta il contrasto nelle zone poco dinamiche ma mantiene sotto controllo il rapporto segnale-rumore. Questo approccio è particolarmente efficace in ambito bio-imaging, dove i dettagli di basso contrasto devono essere resi più evidenti senza degradare la qualità globale dell'immagine.

La normalizzazione dei pixel, infine, agisce sulla distribuzione statistica delle intensità per uniformare scala e offset tra immagini diverse. Nel caso specifico si impiega una normalizzazione di tipo media - varianza (o Z-score), che, per ogni pixel $f(x, y)$, calcola:

$$f'(x, y) = \frac{f(x, y) - \mu}{\sigma}$$

con μ e σ rispettivamente media e deviazione standard dell'intensità dei pixel dell'immagine o del dataset. Sottraendo la media si rimuove il bias di luminosità globale (centering), mentre la divisione per σ rende la varianza unitaria, producendo feature con scala confrontabile e gradienti più stabili durante l'addestramento della rete neurale.

1.3.2 Segmentazione e Tecniche classiche

La segmentazione delle immagini è il processo con cui un'immagine $f(x, y)$ viene suddivisa in regioni omogenee, ognuna associata a un oggetto o a una parte significativa della scena. Costituisce lo stadio che collega l'elaborazione a basso livello all'analisi quantitativa e alle successive fasi di classificazione. Le tecniche classiche della segmentazione prevedono principalmente il tresholding, cioè la

definizione di una o più soglie sull'intensità per separare i pixel di foreground (in primo piano) da quelli di background (lo sfondo), spesso combinato con operazioni morfologiche (ad esempio la rimozione di piccoli oggetti) per ottenere una maschera più pulita e coerente con la geometria delle strutture di interesse. Dato un valore di soglia T, il tresholding globale definisce un'immagine binaria $g(x, y)$ come:

$$g(x, y) = \begin{cases} 1, & f(x, y) \geq T \\ 0, & f(x, y) < T \end{cases}$$

dove i pixel con valore 1 rappresentano un soggetto e quelli con valore 0 lo sfondo.

1.3.3 Reti neurali convoluzionali per imaging e bioimaging

Le reti neurali convoluzionali (CNN) sono modelli profondi progettati per elaborare dati strutturati su griglia, come le immagini, sfruttando operazioni di convoluzione locale per estrarre automaticamente feature gerarchiche via via più astratte. Dal punto di vista matematico, ogni layer convoluzionale applica un insieme di filtri $K_k^{(l)}$ all'input $I^{(l-1)}$, producendo mappe di attivazione $I_k^{(l)}$secondo:

$$I_k^{(l)}(x, y) = \sigma \left(\sum_{i,j} I^{(l-1)} (x + i, y + j) \, K_k^{(l)}(i, j) + b_k^{(l)} \right)$$

dove σ è una funzione di attivazione non lineare e $b_k^{(l)}$ è il bias del filtro. Grazie alla condivisione dei pesi e alla località dei kernel, le CNN riducono drasticamente il numero di parametri rispetto ad una rete completamente connessa, mantenendo la struttura spaziale dell'immagine e risultando particolarmente adatte a compiti di classificazione, localizzazione e segmentazione.

Nel contesto dell'imaging biomedico, le CNN hanno mostrato prestazioni spesso comparabili o superiori a quelle degli esperti umani in numerosi task, tra cui classificazione di patologie su radiografie, TAC e RM, rilevamento di tumori, analisi di immagini istopatologiche e segmentazione automatica di organi o lesioni. In bioimaging cellulare e tissutale, modelli come U-Net e varianti derivate costituiscono lo standard de facto per la segmentazione, grazie alla struttura encoder–decoder con

connessioni skip che combinano informazione di basso livello (dettagli locali) e alto livello (contesto globale) per predire, per ogni pixel, una probabilità di appartenenza alla classe di interesse. In questo scenario si inseriscono anche architetture specializzate come Cellpose o SAM, che modellano esplicitamente la morfologia cellulare e forniscono una segmentazione *instance-level* robusta a variazioni di contrasto e forma, diventando uno strumento centrale nei workflow di bioimaging moderni.

1.3.4 Modelli per la segmentazione

Cellpose "classico" utilizza un tipo di rete che, data l'immagine in input, predice per ogni pixel un campo di flusso vettoriale e una mappa di probabilità; le linee di flusso vengono poi seguite per raggruppare i pixel in singole istanze di cellula, fornendo una segmentazione instance-level accurata e generalista su molti tipi di microscopia. Cellpose-SAM mantiene questo schema di output ma sostituisce l'encoder convoluzionale originale con il backbone Transformer pre-addestrato di SAM, adattato al framework Cellpose: in pratica sfrutta la capacità di SAM di catturare contesto globale e pattern molto diversi e la combina con il decoder specifico di Cellpose per flussi e maschere.

La scelta di usare Cellpose-SAM invece di Cellpose tradizionale è motivata dalla maggiore capacità di generalizzazione: il modello è stato reso esplicitamente robusto a permutazioni dei canali, variazioni di scala delle cellule, rumore, sfocatura isotropa/anisotropa e downsampling, riducendo la necessità di riaddestramento o di tuning aggressivo quando si cambiano condizioni sperimentali. Studi recenti mostrano che Cellpose-SAM raggiunge prestazioni superiori all'accordo medio tra annotatori umani ("superhuman generalization") e si integra pienamente nell'ecosistema Cellpose (fine-tuning, human-in-the-loop, 3D, image restoration), rendendolo particolarmente adatto come blocco di base per pipeline di bioimaging che devono essere riutilizzabili su dataset eterogenei come quello considerato in questa tesi.

1.4 Modelli di classificazione supervisionata

In questa sezione vengono descritti i modelli di classificazione supervisionata utilizzati per distinguere automaticamente tra cellule di buona e cattiva qualità a partire dalle feature morfologiche e di intensità estratte dalla segmentazione, e successivamente per distinguere le varie categorie. In particolare, sono stati adottati due approcci complementari: la regressione logistica (Logistic Regression), che fornisce un modello lineare interpretabile in termini di contributo delle singole feature alla probabilità di essere "buone" e la Random Forest, modello ad alberi multipli basato su *bagging* (campionamento bootstrap e aggregazione delle predizioni) che consente di catturare relazioni non lineari e interazioni tra parametri.

1.4.1 Logistic regression

La regressione logistica è un modello di classificazione binaria che stima direttamente la probabilità che un'osservazione appartenga alla classe positiva (ad esempio, cellula "good"), dato un vettore di feature $x \in \mathbb{R}^n$. Dal punto di vista matematico, il modello assume una relazione lineare tra le feature e il *logit* della probabilità:

$$log \frac{P(Y = 1 \mid x)}{P(Y = 0 \mid x)} = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$$

Dove i coefficienti β_i sono stimati massimizzando la verosimiglianza (con eventuale regolarizzazione L2 per limitare l'overfitting). Applicando la funzione sigmoide

$$\sigma(z) = 1/(1 + e^{-z})$$

si ottiene direttamente $P(Y = 1 \mid x)$, che può essere confrontata con una soglia (nel nostro caso 0.8) per assegnare l'etichetta "*good*" o "*bad*", rendendo il modello interpretabile in termini di contributo di ciascuna feature. La regressione logistica può essere vista come un classificatore lineare nello spazio delle feature: ogni coefficiente β_i misura l'effetto di un incremento unitario della feature x_i sul log-odds della classe positiva, permettendo di quantificare quali parametri influenzano maggiormente la probabilità che una cellula venga considerata di buona qualità.

1.4.2 Random Forest

La Random Forest è un modello di classificazione basato su un ensemble di alberi decisionali, addestrati su campioni bootstrap del dataset e con selezione casuale di sottoinsiemi di feature a ogni split. Ogni albero è un classificatore ad alta varianza, costruito mediante suddivisioni ricorsive dello spazio delle feature in regioni omogenee rispetto all'etichetta; la Random Forest combina le predizioni di molti alberi indipendenti tramite voto di maggioranza, riducendo la varianza complessiva del modello grazie al principio del bagging.

Formalmente, si generano B alberi T_b $(b = 1, \dots, B)$, ciascuno addestrato su un campione bootstrap e, a ogni nodo, la scelta della feature di split avviene tra un sottoinsieme casuale di dimensione m_{try}. Per una nuova osservazione x, la probabilità stimata della classe positiva è la media delle probabilità restituite dai singoli alberi, e la classe finale è il risultato del voto di maggioranza. Questa doppia sorgente di casualità (bootstrap sui dati e selezione casuale delle feature) decorrela gli alberi, portando a un classificatore complessivamente robusto al rumore e meno soggetto a overfitting rispetto al singolo albero. Inoltre, la Random Forest fornisce misure di importanza delle feature (ad esempio basate sulla riduzione dell'impurità o sul permutation importance), utili per identificare quali parametri morfologici e radiometrici contribuiscono maggiormente a discriminare tra cellule buone e cattive.

1.5 Tecniche di riduzione dimensionale

Per interpretare la struttura del dataset di cellule nello spazio delle feature e visualizzare possibili cluster o separazioni tra categorie, sono state utilizzate due tecniche di riduzione dimensionale: la *Principal Component Analysis* (PCA) e il *t-Distributed Stochastic Neighbor Embedding* (t-SNE). La PCA fornisce una trasformazione lineare che proietta i dati su una base ortogonale di componenti principali ordinati per varianza spiegata, mentre t-SNE è un metodo non lineare che costruisce un embedding bidimensionale preservando le relazioni di vicinato.

1.5.1 Principal Component Analysis

La Principal Component Analysis (PCA) è una trasformazione lineare che, dato un insieme di dati centrati $x \in \mathbb{R}^n$, individua una nuova base ortogonale di vettori $\{u_1, \dots, u_n\}$ (le componenti principali) tali che la prima componente massimizzi la varianza proiettata, la seconda massimizzi la varianza residua soggetta a ortogonalità, e così via. Operativamente, si calcola la matrice di covarianza Σ del dataset e si risolve il problema agli autovalori $\Sigma u_i = \lambda_i u_i$; gli autovettori u_i sono le direzioni principali e gli autovalori λ_i quantificano la varianza spiegata lungo ciascuna direzione. Ordinando gli autovalori in senso decrescente e scegliendo le prime k componenti, si ottiene una rappresentazione ridotta $z \in \mathbb{R}^k$ che conserva una frazione della varianza totale pari a $\sum_{i=1}^{k} \lambda_i \,/\, \sum_{j=1}^{n} \lambda_j$.

Questa riduzione dimensionale consente di:

- comprimere l'informazione in poche variabili non correlate,
- filtrare il rumore associato a direzioni di bassa varianza,
- produrre mappe bidimensionali (PC1–PC2) utili per visualizzare eventuali cluster o separazioni tra cellule "good/bad" o tra diverse categorie funzionali.

1.5.2 t-Distributed Stochastic Neighbor Embedding

È una tecnica di riduzione dimensionale non lineare progettata per la visualizzazione di dati ad alta dimensionalità in 2D o 3D. L'idea di base è costruire, nello spazio originale, una distribuzione di probabilità sulle coppie di punti in modo che punti vicini abbiano probabilità elevata di essere scelti come "vicini", mentre punti lontani abbiano probabilità trascurabile; successivamente, si cerca un embedding a bassa dimensione in cui una distribuzione analoga (definita tramite una distribuzione t) approssimi nel modo migliore quella originale, minimizzando una divergenza di tra le due distribuzioni.

Il parametro chiave è la perplexity, che controlla il numero effettivo di vicini considerati e quindi il compromesso tra struttura locale e globale: valori piccoli enfatizzano i dettagli locali, mentre valori grandi tendono a evidenziare pattern più globali, pur al prezzo di possibili distorsioni nelle distanze tra cluster. Poiché t-SNE non preserva le distanze metriche ma le relazioni di vicinato, le mappe ottenute vanno interpretate principalmente in termini di forma e separazione dei gruppi, più che di posizione o scala assoluta; ciò lo rende particolarmente adatto a visualizzare, nello spazio bidimensionale, le strutture latenti delle feature delle cellule e a verificare se cellule di diverse categorie immunologiche tendono a formare cluster distinti.

Capitolo 2 Metodologia

2.1 Dataset

In questo lavoro sono stati considerati diversi dataset, utilizzati in fasi distinte di training e di test della pipeline di analisi. Il dataset principale, impiegato per addestrare i modelli e definire la maggior parte delle statistiche, è composto da tre categorie di linfociti T: Naive (mai attivati), Attivi (…) ed Esausti (…), che rappresentano stati funzionali progressivi dall'assenza di stimolazione antigenica fino a condizioni di attivazione prolungata e perdita parziale di efficienza effettrice.

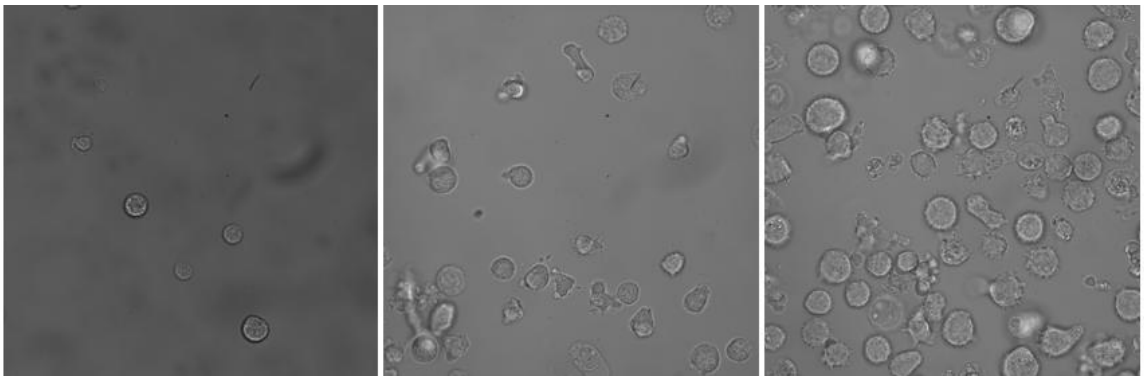


Figura 2.1 Linfociti T: 1) Naive; 2) Activated; 3) Exhausted

Per valutare la capacità del programma di riconoscere correttamente le categorie, sono stati utilizzati invece due dataset caratterizzati da condizioni sperimentali più "incerte": nel primo, i linfociti sono stati attivati (in vitro) solo per mezza giornata, rendendoli quindi associabili allo stato naive di partenza; nel secondo invece troviamo dei linfociti attivati una prima volta e riattivate dopo sei giorni, producendo un fenotipo intermedio tra cellule ancora effettivamente attive ed elementi che iniziano a mostrare segni di esaustività. Rendendo questi dataset di test un'ottima opportunità per verificare la sensibilità del modello.

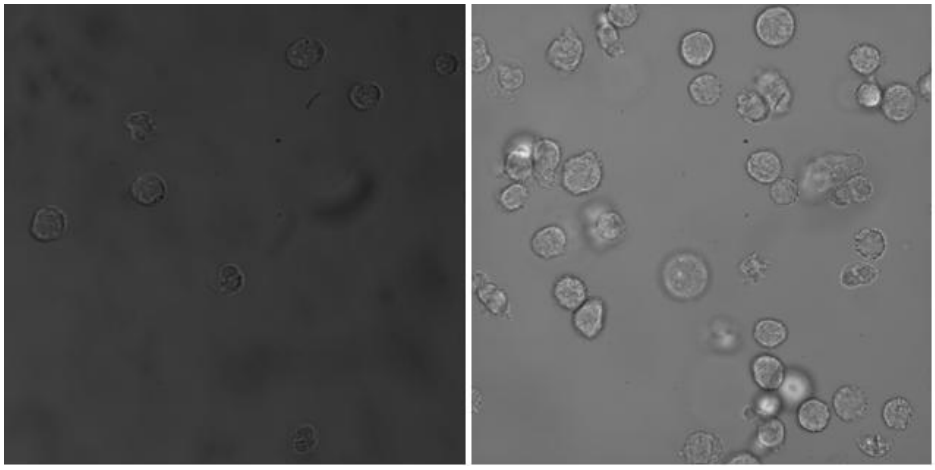


Figura 2.2 1) Linfociti ca. Naive; 2) Linfociti Attivi / Esausti

2.2 Ambiente di calcolo e librerie

L'ambiente di calcolo costituisce un elemento fondamentale del lavoro svolto, poiché determinante delle risorse hardware disponibili e gli strumenti software impiegati per implementare le pipeline di pre-processing, segmentazione e analisi statistica delle cellule. In questa sezione verranno descritti, da un lato l'utilizzo di Google Colab e delle sue risorse GPU per l'esecuzione dei notebook di deep learning, e, dall'altro, il linguaggio che sfrutta, quindi Python e le librerie utilizzate con quest'ultimo.

2.2.1 Google Colab

Per lo sviluppo del codice è stato dunque utilizzato Google Colab come ambiente principale di calcolo, in quanto fornisce notebook Jupyter eseguibili nel cloud con accesso gratuito (ma offre anche opzioni a pagamento) a risorse GPU senza necessità di configurare un hardware locale. Questa scelta ha permesso di addestrare e valutare in modo efficiente il modello di segmentazione Cellpose-SAM e i classificatori basati su machine learning, sfruttando la parallelizzazione intrinseca delle GPU per elaborare grandi batch di immagini e gestire pipeline computazionalmente onerose.

Figura 2.3 Google Colab Logo

Le sessioni sono state eseguite selezionando un Runtime con GPU (come NVIDIA T4 o la v5e-1 TPU per la versione gratuita e altre come A100 GPU, H100 GPU per i piani a pagamento) e montando Google Drive come spazio di archiviazione per dataset, notebook e risultati, cosi da garantire tracciabilità degli esperimenti e possibilità di riproduzione delle analisi anche a distanza di tempo e in diverse sessioni. I limiti tipici del servizio, come una durata limite per le sessioni, policy di "fair use" delle GPU (viene richiesto di non collegarsi ad una GPU se non per necessità) e disconnessione del Runtime in caso di inattività (con conseguente perdita dei dati locali), sono stati gestiti strutturando il codice in blocchi indipendenti e salvando ogni risultato sul drive, rendendo così il workflow meno sensibile alle eventuali interruzioni.

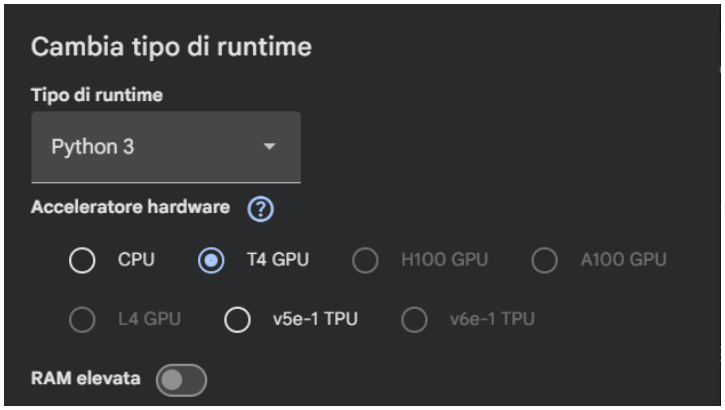


Figura 2.4 GPU offerte da Google Colab

2.2.2 Python e Librerie utilizzate

L'intera pipeline è stata sviluppata in Python, sfruttandone l'ecosistema scientifico per integrare in modo compatto pre-processing, segmentazione, estrazione delle feature e modellazione statistica. Questo linguaggio è in grado di gestire da un lato l'accesso dei dati e l'organizzazione dei risultati in strutture tabellari (principalmente tramite la libreria "pandas"), dall'altro permette di organizzare le varie fasi della pipeline all'interno del notebook eseguibile in Colab.

Per il pre-processing e l'image processing classico sono state impiegate librerie quali "Open CV" e "scikit-image", utilizzate per operazioni su immagini in scala di grigi,

rimozione del background (“unsharp mask”), CLAHE, normalizzazione dei pixel e morfologia matematica. La segmentazione cellulare è stata affidata a Cellpose-SAM, richiamato tramite l’API “CellposeModel” su immagini normalizzate, mentre l’estrazione delle feature geometriche e radiometriche per ogni cellula (es. area, perimetro, rotondezza…) è stata implementata combinando gli strumenti di “scikit-image” con operazioni vettoriali in “NumPy”.

Per l’analisi statistica e la classificazione sono state utilizzate le librerie machine learning di “scikit-learn” quali: “StandardScaler” e “SimpleImputer” per la standardizzazione e la gestione dei dati mancanti (se in una tabella mancano dei valori, questi verranno aggiunti con la media/mediana dei valori presenti nella propria colonna), “LogisticRegression” e “RandomForestClassifier” per la predizione automatica della qualità delle cellule e per la categorizzazione, oltre a strumenti usati per la validazione (train-test, split, learning curve e classification report) e per la riduzione dimensionale (PCA e t-SNE). La visualizzazione dei risultati, come curve di apprendimento, mappe PCA e t-SNE e distribuzione delle feature, è stata gestita tramite le librerie “Matplotlib” e “Seaborn”, consentendo un’interpretazione qualitativa delle prestazioni dei modelli e della struttura dei dati.

```
from cellpose import models
import matplotlib.pyplot as plt
from IPython.display import display
from skimage.morphology import remove_small_objects
from skimage.measure import label, regionprops
from skimage.feature import peak_local_max
from skimage.segmentation import watershed
from scipy import ndimage as ndi
import numpy as np
import random
import glob
import cv2
import pandas as pd
import os
from skimage.measure import regionprops, label
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.impute import SimpleImputer
from sklearn.model_selection import learning_curve
import shutil
from sklearn.decomposition import PCA
from mpl_toolkits.mplot3d import Axes3D
from sklearn.manifold import TSNE
from sklearn.metrics import classification_report, confusion_matrix
import joblib
cp_model = models.CellposeModel(gpu=True, pretrained_model='cpsam')
```

Figura 2.5 Librerie installate nel codice

2.3 Pipeline di Image processing

In questo paragrafo viene descritta la pipeline di analisi sviluppata in Python per passare dalle immagini grezze alle misure quantitative sui linfociti. Dopo aver introdotto nel Capitolo 1 i concetti teorici di *image processing*, segmentazione e reti neurali convoluzionali, qui si mostra come tali idee sono state tradotte in pratica nel codice: dalla fase di pre-processing (scala di grigi, rimozione del background, CLAHE e normalizzazione), alla segmentazione automatica con Cellpose-SAM, fino all'estrazione delle feature morfologiche e fotometriche e alla successiva classificazione statistica delle cellule. Quasi tutte le fasi di elaborazione delle immagini sono state implementate in una singola funzione.

2.3.1 Pre-processing in Python

La pre-elaborazione delle immagini è un punto chiave del codice che sottolinea l'idea globale, ovvero quella di rendere il codice adatto a diversi tipi di immagini, senza la necessità di modificarle opportunamente. In particolare ci soffermiamo su:

- Caricamento in scala di grigi "*cv2.imread(..., cv2.IMREAD_GRAYSCALE)*" converte l'immagine originale in un singolo canale di intensità, riducendo la dimensionalità del problema e rendendo più stabile il calcolo dei livelli di grigio per la segmentazione.

```
# 1) Load grayscale image
img_gray = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
if img_gray is None:
    raise FileNotFoundError(f"Image not found or unreadable: {img_path}")
```

- Filtro Gaussiano *("cv2.GaussianBlur")*: applica una convoluzione con kernel gaussiano che attenua il rumore ad alta frequenza e le piccole fluttuazioni locali, preservando le strutture più grandi che corrispondono alle cellule. Rimozione del background *(unsharp mask; cv2.addWeighted tra immagine originale e sfocata)*: enfatizza i dettagli ad alta frequenza sottraendo una versione sfocata dell'immagine, riducendo i gradienti lenti dovuti all'illuminazione non uniforme e migliorando il rapporto cellula-sfondo.

```
# 2) Background removal (unsharp mask)
img_f = img_gray.astype(np.float32)
blur = cv2.GaussianBlur(img_f, (0, 0), 4)
img_bg_removed = cv2.addWeighted(img_f, 1.9, -blur, 0.5, 0)
```

- Equalizzazione locale del contrasto (CLAHE) *(cv2.createCLAHE):* esegue un'equalizzazione dell'istogramma su piccole regioni, limitando il contrasto tramite il parametro "*clipLimit*" e aumentando la visibilità di strutture poco contrastate senza amplificare eccessivamente il rumore.

```
# 3) Local contrast enhancement (CLAHE)
img_bg_removed = np.clip(img_bg_removed, 0, 255).astype(np.uint8)
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
img_eq = clahe.apply(img_bg_removed)
```

- Normalizzazione dei pixel (Normalizzazione Z-score): riporta le intensità ad un intervallo controllato o a distribuzioni standardizzate ($x' = (x - \mu)/\sigma$), così da rendere confrontabili le immagini tra loro e numericamente ben condizionato l'input al modello di segmentazione. In questo lavoro, μ e σ sono state calcolate in una funzione separata, ottenendo dunque la media e la deviazione standard delle intensità dei pixel presi da un campione di immagini del dataset originale.

```
# 4) Normalization using dataset mean and std
img_norm = (img_eq.astype(np.float32) - mu) / (sigma + 1e-8)
```

2.3.2 Segmentazione con Cellpose-SAM e post-processing in Python

La segmentazione con Cellpose-SAM rappresenta il fulcro della pipeline, poiché traduce le immagini pre-processate in maschere etichettate di ogni singola cellula sulle quali vengono poi calcolate tutte le feature morfologiche e fotometriche. A differenza di un tradizionale tresholding, la separazione tra sfondo e oggetti è affidata al modello di deep learning Cellpose-SAM, che combina il backbone Transformer di SAM con il decoder di Cellpose, producendo per ogni pixel dei campi di flusso e mappe di probabilità da cui si ricavano automaticamente le istanze cellulari.

Operativamente, il codice carica il modello “CellposeModel(gpu=True, pretrained_model=’cpsam’)” e lo applica alle immagini normalizzate in scala di grigi, ottenendo una prima maschera di segmentazione (maskcp) insieme ai flussi e agli “styles”. Sulla maschera verrà poi eseguito un post-processing deterministico:

- Rimozione degli oggetti troppo piccoli tramite “remove_small_object” per eliminare rumore e frammenti non biologicamente rilevanti.
- Operazioni morfologiche con kernel ellittici “cv2.morphologyEx” per lisciare i bordi delle cellule.
- Applicazione di un algoritmo di Watershed sui “blob” di area molto grande, in modo da separare cellule aderenti o parzialmente sovrapposte partendo dalla mappa di distanza e dai massimi locali. Il risultato è una mappa di etichette finali in cui ogni regione connessa corrisponde, per quanto possibile, ad una singola cellula.

```
# 5) Cellpose-SAM segmentation (monochannel image -> channels=[0,0])
masks_cp, flows, styles = cp_model.eval(
    img_norm,
    channels=[0, 0]
)

# 6) Morphological cleanup:
#    - remove small objects (noise)
#    - smooth edges using morphological opening
masks_cp = remove_small_objects(masks_cp.astype(bool), min_size=3000)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
masks_cp = cv2.morphologyEx(
    masks_cp.astype(np.uint8),
    cv2.MORPH_OPEN,
    kernel
)

# 7) Watershed-based splitting of large blobs to separate touching cells
lab0 = label(masks_cp.astype(bool))  # initial labels of connected components
lab_final = np.zeros_like(lab0, dtype=np.int32)
current_label = 1

for r in regionprops(lab0):
    # mask of the current connected component (blob)
    blob_mask = (lab0 == r.label)

    # If the blob is not too large, keep it as a single cell
    if r.area < 18000:  # threshold may need tuning for your images
        lab_final[blob_mask] = current_label
        current_label += 1
        continue

    # For large blobs, compute distance transform
    distance = ndi.distance_transform_edt(blob_mask)
```

```python
    # Find local maxima as potential cell centers
    coords = peak_local_max(
        distance,
        footprint=np.ones((50, 50)),  # minimum separation between centers
        labels=blob_mask
    )

    # Build marker image for watershed
    local_max = np.zeros_like(distance, dtype=bool)
    if coords.size > 0:
        local_max[coords[:, 0], coords[:, 1]] = True
    markers = ndi.label(local_max)[0]

    # Apply watershed to split the blob into multiple labels
    ws = watershed(-distance, markers, mask=blob_mask)

    # Assign new labels to each watershed region
    for k in np.unique(ws)[1:]:  # skip background 0
        lab_final[ws == k] = current_label
        current_label += 1

masks_cp = lab_final  # final labeled mask

return img_gray, masks_cp, img_eq
```

Questa combinazione di modello neurale e regole morfologiche consente di ottenere una segmentazione robusta a variazioni di contrasto, rumore e dimensione cellulare, sfruttando da un lato la capacità di generalizzazione di Cellpose-SAM e dall'altro la possibilità di controllare esplicitamente, tramite soglie e parametri geometrici, la sensibilità della separazione tra cellule in contatto.

2.4 Estrazione delle feature

L'estrazione delle feature rappresenta il passaggio in cui le maschere di segmentazione vengono trasformate in descrittori numerici delle singole cellule, rendendo possibile un'analisi quantitativa e l'utilizzo di modelli statistici o di machine learning. A partire dalla maschera etichettata da Cellpose-SAM e dall'immagine equalizzata, il codice costruisce, per ogni cellula k, un vettore di feature f_k che combina misure di forma (area, perimetro, rotondezza (circularity), ecc.) e misure fotometriche (intensità media, contrasto con lo sfondo, ecc.). Questi descrittori costituiscono la base sia per la caratterizzazione morfologica dei linfociti, sia per la definizione di indici sintetici di qualità della segmentazione, utilizzati in seguito per addestrare i classificatori "good/bad".

2.4.1 Feature morfologiche

Le feature morfologiche sono state calcolate direttamente a partire dalla maschera etichettata prodotta da Cellpose-SAM, utilizzando la funzione "*regionprops"* di *skimage.measure,* che fornisce per ogni etichetta un insieme coerente di proprietà geometriche dell'oggetto. Dal punto di vista implementativo, il codice itera sulle regioni restituite da *regionprops(lab, intensity_image = img_eq)* e costruisce per ciascuna cellula un dizionario di misure che viene poi raccolto in un DataFrame pandas, assicurandosi una corrispondenza uno-a-uno tra ID della cellula nella maschera (label) e riga della tabella delle feature.

Le grandezze affrontate in questo contesto sono:

- *Area* e *perimetro*, che quantificano rispettivamente la dimensione e la lunghezza del contorno della cellula in pixel. Questi valori vengono letti da *regionprops* e costituiscono la base per altri indici derivati.
- *Circularity*, calcolata nel codice come: $4\pi \frac{area}{perimetro^2+10^{-8}}$, dove un valore pari a 1 indica una cellula perfettamente circolare, mentre valori più bassi corrispondono a profili allungati o irregolari. L'aggiunta del termine 10^{-8} al denominatore evita instabilità numeriche per perimetri molto piccoli.
- *Solidity*, definita come rapporto tra area reale e area del suo inviluppo convesso. Nel codice viene letta direttamente da *regionprops* e utilizzata per penalizzare cellule con contorni molto frastagliati o "bucati", indicative di segmentazioni poco affidabili.
- *Eccentricità*, che deriva dagli autovalori del tensore d'inerzia e misura quanto la cellula è allungata (0 per un cerchio, valori prossimi a 1 per forme molto allungate e strette). È calcolata internamente da *regionprops* e riportata senza ulteriori trasformazioni.
- *Assi maggiore e minore*, che rappresentano l'asse maggiore e minore dell'ellisse equivalente) e relativo *aspect ratio* (rapporto tra i due), che descrivono in modo intuitivo l'elongazione della cellula e sono

particolarmente utili per confrontare categorie di linfociti con morfologie diverse.

- *Extent*, definita come rapporto tra area e area del bounding box minimo (il rettangolo che inscrive la cellula). Questa misura evidenzia oggetti che occupano solo una piccola frazione del riquadro, potenzialmente rumore o segmentazioni tronche.

```
cells = []

for r in regions:
    # Basic shape features
    area = r.area
    perimeter = r.perimeter

    circularity = (4 * np.pi * area) / (perimeter**2 + 1e-8)
    solidity = r.solidity
    eccentricity = getattr(r, "eccentricity", np.nan)
    major_axis = getattr(r, "major_axis_length", np.nan)
    minor_axis = getattr(r, "minor_axis_length", np.nan)
    if minor_axis == 0:
        aspect_ratio = np.nan
    else:
        aspect_ratio = major_axis / minor_axis

    # Extent: area / area of bounding box
    minr, minc, maxr, maxc = r.bbox
    bbox_area = (maxr - minr) * (maxc - minc)
    extent = area / (bbox_area + 1e-8)
```

```
    cells.append({
        "label": r.label,        # Cell ID in the mask
        # Shape features
        "area": area,
        "perimeter": perimeter,
        "circularity": circularity,
        "solidity": solidity,
        "eccentricity": eccentricity,
        "major_axis": major_axis,
        "minor_axis": minor_axis,
        "aspect_ratio": aspect_ratio,
        "extent": extent,
```

2.4.2 Feature fotometriche

Queste sono state progettate per descrivere il contenuto di intensità della cellula e il suo rapporto con lo sfondo, oltre a fornire indici sintetici di visibilità e qualità della segmentazione. Come per le feature di forma, il calcolo parte da regionprops(lab, intensity_image=img_eq), in modo che per ogni regione siano disponibili le grandezze radiometriche standard (mean_intensity, max_intensity, min_intensity, ecc.) direttamente sulla versione equalizzata dell'immagine. Su queste misure "di base" il codice costruisce poi feature derivate specifiche per il problema dei linfociti.

I parametri considerati sono dunque:

- *Mean_int, max_int e min_int*, letti da regionprops e che rappresentano rispettivamente l'intensità media, massima e minima dei pixel nella cellula.
- *Std_int,* calcolata come deviazione standard dei pixel della cellula, e *median_int,* come mediana di tali valori. Queste grandezze quantificano quanto il segnale intracellulare è omogeneo o eterogeneo, aspetto importante per distinguere cellule ben segmentate da regioni che includono porzioni di sfondo o artefatti.
- Per caratterizzare lo sfondo locale e il contrasto cellula-sfondo, il codice costruisce in modo esplicito un anello attorno a ciascuna cellula:
    - Crea una maschera binaria dalla cellula (*cellmask*).
    - La dilata con un kernel ellittico (*cv2.dilate*) e sottrae la maschera originale, ottenendo un anello di pixel circostanti (*ring*).
    - Campiona da img_eq solo i pixel dell'anello (*ringvals*).
- *Bg_mean* e *bg_median* (media e mediana di sfondo) vengono calcolati a partire da ringvals, e quando l'anello contiene pixel sufficienti, viene calcolata anche *edge_int_mean* (intensità media di bordo).
- A partire dai valori appena citati, definiamo:

- *Contrast_mean = mean_int – bg_mean,* ovvero il contrasto assoluto medio tra la cellula e lo sfondo.
- *contrast_max = max_int – bg_mean,* ovvero il contrasto massimo tra cellula e sfondo.
- *Ratio_mean = mean_int – (bg_mean +* 10^{-8}*),* che corrisponde a un contrasto relativo analogo al contrasto percentuale usato in microscopia.

```
# Local background ring around the cell
cell_mask = np.zeros_like(masks_cp, dtype=np.uint8)
cell_mask[lab == r.label] = 1

kernel_size = 2 * bg_ring + 1
kernel = cv2.getStructuringElement(
    cv2.MORPH_ELLIPSE,
    (kernel_size, kernel_size)
)
dilated = cv2.dilate(cell_mask, kernel, iterations=1)
ring = (dilated == 1) & (cell_mask == 0)

ring_vals = img_eq[ring]
if ring_vals.size > 0:
    bg_mean = float(np.mean(ring_vals))
    bg_median = float(np.median(ring_vals))
else:
    bg_mean = np.nan
    bg_median = np.nan

# Derived intensity/contrast features
contrast_mean = mean_int - bg_mean if not np.isnan(bg_mean) else np.nan
contrast_max = max_int - bg_mean if not np.isnan(bg_mean) else np.nan
ratio_mean = mean_int / (bg_mean + 1e-8) if not np.isnan(bg_mean) else np.nan

# Edge intensity (mean intensity on the ring)
if ring_vals.size > 0:
    edge_int_mean = float(np.mean(ring_vals))
else:
    edge_int_mean = bg_mean
```

2.4.3 Feature combinate

Su questo insieme di feature viene costruita una coppia di feature combinate:

- *Visibility,* definita nel codice come prodotto tra contrast_mean e solidity, per assegnare valori elevati a cellule sia ben contrastate rispetto allo sfondo sia morfologicamente compatte.

- *Quality score,* ottenuto combinando linearmente (e scalando) contrast_mean, circularity e solidity con dei pesi fissati (1.0 per il contrato e 0.5 per le due morfologiche riportate nel range 0-100), e impostato a NaN (Not a Number) per valori palesemente fuori scala.

Queste scelte implementative rendono il *quality score* un indicatore deterministico e riproducibile della bontà della segmentazione (almeno prima delle statistiche), che ingloba in un unico valore sia aspetti radiometrici (quanto la cellula è visibile rispetto allo sfondo) sia aspetti geometrici (quanto la sua forma è regolare).

```python
# Combined "visibility" and "quality" scores
visibility = contrast_mean * solidity if not np.isnan(contrast_mean) else np.nan
quality_score = (
    1.0 * (contrast_mean if not np.isnan(contrast_mean) else 0.0)
    + 0.5 * (circularity if not np.isnan(circularity) else 0.0) * 100
    + 0.5 * (solidity if not np.isnan(solidity) else 0.0) * 100
)
```

```python
        cells.append({
            "label": r.label,        # Cell ID in the mask
            # Shape features
            "area": area,
            "perimeter": perimeter,
            "circularity": circularity,
            "solidity": solidity,
            "eccentricity": eccentricity,
            "major_axis": major_axis,
            "minor_axis": minor_axis,
            "aspect_ratio": aspect_ratio,
            "extent": extent,
            # Intensity features
            "mean_int": mean_int,
            "max_int": max_int,
            "min_int": min_int,
            "std_int": std_int,
            "median_int": median_int,
            # Background / contrast
            "bg_mean": bg_mean,
            "bg_median": bg_median,
            "contrast_mean": contrast_mean,
            "contrast_max": contrast_max,
            "ratio_mean": ratio_mean,
            "edge_int_mean": edge_int_mean,
            # Combined scores
            "visibility": visibility,
            "quality_score": quality_score,
        })
```

2.5 Costruzione dei DataFrame

La costruzione dei DataFrame rappresenta il punto di raccordo tra la fase di segmentazione / estrazione delle feature e tutte le analisi successive (statistiche descrittive, modelli di qualità, classificatori di categoria). In questa sezione viene descritto come, a partire dalle immagini delle tre classi di linfociti (Naive, Activated, Exhausted), il codice genera un'unica tabella strutturata in cui ogni riga corrisponde a una cellula e contiene sia le feature calcolate sia le informazioni di tracciabilità (file di origine, categoria, percorso del crop).

Per ciascuna categoria viene chiamata la funzione *build_dataset_for_category*, che itera su tutte le immagini della cartella, applica la pipeline di segmentazione (*segment_image_cpsam*) e il calcolo dei parametri, e concatena i risultati in un DataFrame per-immagine arricchito con il nome del file di origine, il path (il percorso) dell'immagine e la categoria. In un secondo passaggio, usando la maschera etichettata e le bounding box di regionprops, il codice effettua il *crop* (ritaglio) in scala di grigi di ogni ROI (Region Of Interest) cellulare e lo salva su disco in una sottocartella *crops_raw*, registrando per ogni riga del DataFrame il relativo *crop_raw_path*; questo consente di collegare direttamente ogni vettore di feature alla sua immagine di riferimento. Infine, i DataFrame delle tre categorie vengono concatenati nel DataFrame globale *df_all*, che costituisce l'input standardizzato per il resto della tesi (calcolo di statistiche per categoria, addestramento dei modelli good/bad e dei classificatori Naive/Activated/Exhausted).

```
def build_dataset_for_category(img_dir, category_name, mu, sigma):
    rows = []

    crops_root = os.path.join(img_dir, "crops_raw")
    os.makedirs(crops_root, exist_ok=True)


    for fname in os.listdir(img_dir):
        if not fname.lower().endswith((".png", ".jpg", ".jpeg", ".tif", ".tiff")):
            continue
        path = os.path.join(img_dir, fname)

        img_gray, masks_cp, img_eq = segment_image_cpsam(path, mu, sigma)
        df_img = parameters(img_gray, masks_cp, img_eq)

        df_img["filename"] = fname
        df_img["path"] = path
        df_img["category"] = category_name
        rows.append(df_img)
```

2.6 Modelli di analisi supervisionata delle cellule

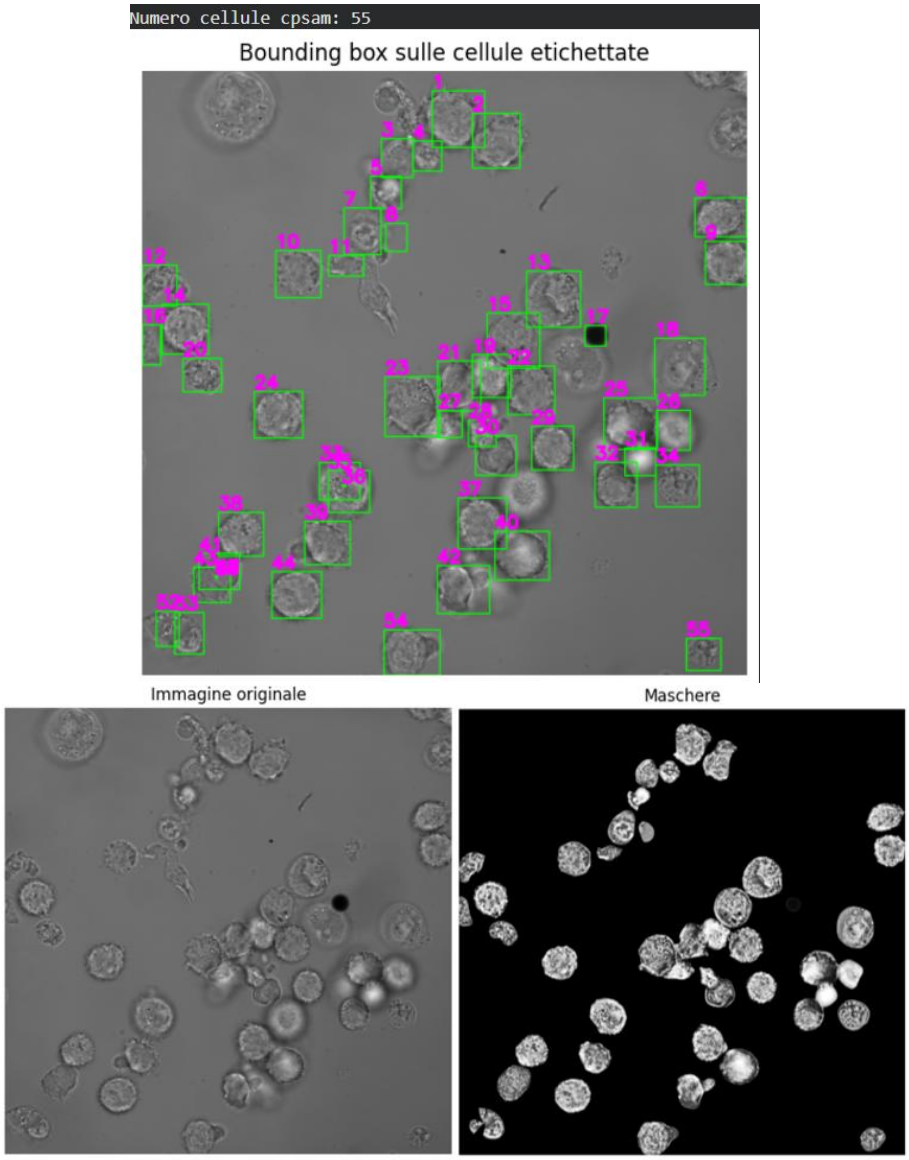
Per utilizzare in modo affidabile le misure estratte sulle singole cellule è necessario introdurre modelli di classificazione supervisionata che operino sia sulla qualità della segmentazione (good/bad) sia sulle categorie funzionali dei linfociti (Naive, Activated, Exhausted). L'idea di fondo è trasformare il *quality score* e le feature morfologiche/fotometriche in etichette discrete: da un lato una variabile binaria che indichi se una cellula è utilizzabile nelle analisi successive oppure debba essere scartata, dall'altro una variabile categoriale che identifichi la classe funzionale di appartenenza. In entrambi i casi, come approfondito nel Capitolo 1, si fa uso di modelli supervisionati quali *Logistic Regression e Random Forest*, che ricevono in ingresso i vettori di feature e restituiscono la probabilità che una cellula appartenga a una certa classe; a supporto dell'interpretazione dei risultati vengono inoltre impiegate tecniche di riduzione dimensionale come *PCA e t-SNE*, utili per visualizzare in uno spazio bidimensionale la distribuzione dei dati e l'eventuale formazione di cluster coerenti con le etichette good/bad e con le categorie funzionali. Nel seguito si descrive come è stato costruito il dataset annotato di riferimento e come questi strumenti vengono poi impiegati nella pipeline.

2.6.1 Etichettatura manuale

Per addestrare modelli in grado di riconoscere automaticamente cellule "buone" o "cattive" ai fini delle analisi, è stato innanzitutto necessario costruire un set di riferimento etichettato manualmente. A tal fine, in un notebook separato ho segmentato diversi campioni di immagini provenienti dalle tre categorie di linfociti e, a partire dai crop delle singole ROI, ho ispezionato visivamente ogni cellula assegnandole un'etichetta *good* (segmentazione completa, contorno corretto, buon contrasto) oppure *bad* (cellula tagliata, unione di più cellule, rumore o bordo poco definito).

Per ciascun campione segmentato sono stati quindi salvati i DataFrame contenenti tutte le feature morfologiche e di intensità, ai quali è stata aggiunta una colonna *manual_label* con il valore "*good*" o "*bad*"; questi diversi DataFrame sono

stati successivamente concatenati in un unico dataset annotato. Questo dataset rappresenta la base per l'addestramento dei modelli supervisionati descritti nei sottoparagrafi successivi e viene utilizzato sia per stimare un classificatore good/bad, sia, in combinazione con le informazioni di categoria funzionale, per costruire in seguito i modelli dedicati alla distinzione tra linfociti Naive, Activated ed Exhausted.




```
# 3) annotazione manuale

category = "Exhausted"
df["manual_label"] = "unknown"
df.loc[df["label"].isin([5,16,18,21,22, 2 ]),  "manual_label"] = "good"
df.loc[df["label"].isin([]), "manual_label"] = "bad"
df_manual = df[df["manual_label"] != "unknown"].copy()
df_manual["Percorso"] = test_path          # percorso o nome file
df_manual["category"] = category
lab = label(masks_cp)

display(df_manual)
```

2.6.2 Applicazione dei modelli di classificazione “Good”/”bad”

Per la classificazione automatica delle cellule in *good* e *bad* sono stati utilizzati, come modelli supervisionati, la Logistic Regression e la Random Forest, introdotte teoricamente nel Capitolo 1. A partire dal dataset annotato descritto nel paragrafo precedente, sono stati costruiti i vettori di feature X e il vettore di etichette binarie y (good = 1, bad = 0), che costituiscono l’input per i modelli. Il dataset è stato suddiviso in training e test tramite *train–test split* stratificato, in modo da mantenere la stessa proporzione good/bad in entrambi i sottoinsiemi e valutare in maniera corretta la capacità di generalizzazione dei classificatori.

Nel caso della Logistic Regression, il modello è stato inserito in una *pipeline* che include un imputatore dei valori mancanti (al posto degli eventuali Nan, inserisce il valore della mediana), una standardizzazione delle feature e una regressione logistica con regolarizzazione L2. Dal modello si ottengono, per ogni cellula, le probabilità $P(\text{good} \mid \mathbf{x})$, che vengono convertite in etichetta binaria applicando una soglia più conservativa del valore standard 0.5 (tra 0.7 e 0.8), così da ridurre il numero di falsi positivi e selezionare come *good* solo le cellule con alta confidenza. Le prestazioni vengono valutate sul set di test mediante classification e curve di apprendimento.

```
log_model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("logreg", LogisticRegression(
        penalty="l2",
        C=0.4,
        solver="lbfgs",
        max_iter=1000,
    )),
])

log_model.fit(X_train, y_train)

y_prob_log = log_model.predict_proba(X_test)[:, 1]  # probability of class "good"
threshold_log = 0.7  # can be tuned (e.g. 0.6, 0.8...)
y_pred_log = (y_prob_log >= threshold_log).astype(int)

print("=== Logistic Regression (good vs bad, test set) ===")
print(classification_report(y_test, y_pred_log, target_names=["bad", "good"]))

# Save Logistic Regression classification report to Excel
log_reg_creport = classification_report(
    y_test,
    y_pred_log,
    target_names=["bad", "good"],
    output_dict=True,
)
df_report_log = pd.DataFrame(log_reg_creport).T
df_report_log.to_excel(
    os.path.join(STATS_DIR, "Creport_logistic_reg_test_goodbad.xlsx"),
    index=True,
```

La Random Forest è stata configurata come un ensemble di alberi decisionali con numero di alberi e profondità massima scelti per bilanciare capacità espressiva e rischio di overfitting. Anche in questo caso il modello è stato inserito in una pipeline con imputazione dei mancanti e standardizzazione, e le probabilità della classe *good* vengono sogliate (ad esempio a 0.8) per ottenere una classificazione più prudente. Oltre alle metriche di accuratezza sul set di test, la Random Forest fornisce una stima dell'importanza delle feature, utile per individuare quali parametri (ad esempio contrasto medio, circularity, solidity) contribuiscono maggiormente alla distinzione tra cellule buone e cattive. I modelli così addestrati sono infine applicati all'intero DataFrame di cellule, permettendo di filtrare automaticamente le segmentazioni di bassa qualità che non verranno considerate nelle analisi successive.

```
rf_model = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler()),
    ("rf", RandomForestClassifier(
        n_estimators=150,
        max_depth=8,
        min_samples_leaf=5,
        random_state=42,
        n_jobs=-1,
    )),
])

rf_model.fit(X_train, y_train)

y_prob_rf = rf_model.predict_proba(X_test)[:, 1]  # probability of class "good"
threshold_rf = 0.8  # stricter threshold for classifying as good
y_pred_rf = (y_prob_rf >= threshold_rf).astype(int)

print("=== Random Forest (good vs bad, test set) ===")
print(classification_report(y_test, y_pred_rf, target_names=["bad", "good"]))
```

Per rendere ancora più selettiva la classificazione e la successiva ispezione nei DataFrame è stato introdotto un combined rank che combina le probabilità/score prodotti da Logistic Regression e Random Forest in un unico indice ordinabile. In questo modo, le cellule classificate come *good* da entrambi i modelli e con maggiore confidenza vengono posizionate ai primi posti del dataframe, facilitando sia il filtraggio delle segmentazioni sia la visualizzazione delle cellule di qualità più elevata.

```
# Simple average ensemble
y_prob_comb_test = 0.5 * y_prob_log_test + 0.5 * y_prob_rf_test
threshold_ensemble = 0.8
y_pred_comb_test = (y_prob_comb_test >= threshold_ensemble).astype(int)

print("=== Ensemble Logistic + Random Forest (good vs bad, test set) ===")
print(classification_report(y_test, y_pred_comb_test, target_names=["bad", "good"]))
```

2.6.3 PCA e t-SNE per l'esplorazione dei cluster di cellule

Per studiare la separazione tra le categorie funzionali (Naive, Activated, Exhausted) è stato innanzitutto costruito un nuovo dataframe contenente solo le cellule con segmentazione giudicata *good* dal modello combinato, così da eliminare il rumore dovuto a contorni errati o artefatti e rendere più marcata la distinzione tra le classi nello spazio delle feature. A partire da questo dataframe sono stati estratti i vettori di feature morfologiche e fotometriche (*feature_cols*) e le corrispondenti etichette di categoria (*category*), sui quali sono state applicate tecniche di riduzione dimensionale lineare (PCA) e non lineare (t-SNE) per ottenere rappresentazioni bidimensionali interpretabili.

Dal punto di vista implementativo, le feature sono state prima imputate con la mediana tramite *SimpleImputer*, quindi standardizzate con *StandardScaler*, e successivamente proiettate con una PCA a sei componenti (*PCA(n_components= 6, random_state=42)*), utilizzando i primi due componenti per le mappe 2D e analizzando varianza spiegata e loadings delle feature. Per valutare la separazione tra categorie, la PCA è stata applicata sia sull'intero insieme di cellule buone (tutte e tre le classi), sia su coppie di categorie (*Naive vs Activated, Naive vs Exhausted, Activated vs Exhausted*), costruendo di volta in volta dataframe filtrati per coppia.

In parallelo, sugli stessi vettori scaled è stato eseguito t-SNE (*TSNE(n_components=2, perplexity=30, learning_rate=200, n_iter=1000, random_state=42)*), generando mappe bidimensionali per le stesse combinazioni di classi; queste proiezioni, pur non entrando nella fase di classificazione, permettono di visualizzare eventuali cluster compatti e di verificare qualitativamente se le categorie funzionali dei linfociti occupano regioni distinte nello spazio delle feature.

```
# Save list of "good" cells only
df_good_cells = df_all_pred_with_crops[df_all_pred_with_crops["label_comb"] == "good"].copy()
df_good_cells.to_excel(os.path.join(STATS_DIR, "df_good_cells.xlsx"), index=False)
```

```python
# Feature matrix and category labels for good cells
X_good = df_good_cells[feature_cols].values
y_cat = df_good_cells["category"].values

# Impute missing values and standardize
imputer = SimpleImputer(strategy="median")
X_good_imp = imputer.fit_transform(X_good)

scaler = StandardScaler()
X_good_scaled = scaler.fit_transform(X_good_imp)
```

```python
pca = PCA(n_components=8, random_state=42)
X_pca = pca.fit_transform(X_good_scaled)

# Scatter plot PC1 vs PC2 by category
plt.figure(figsize=(6, 5))
for cat in np.unique(y_cat):
    mask = (y_cat == cat)
    plt.scatter(
        X_pca[mask, 0],
        X_pca[mask, 1],
        s=20,
        alpha=0.7,
        label=cat,
    )
```

```python
tsne = TSNE(
    n_components=2,
    perplexity=30,
    learning_rate=200,
    n_iter=1000,
    random_state=42,
)

X_tsne = tsne.fit_transform(X_good_scaled)
```

.....

```python
pairs = [
    ("Naive", "Activated"),
    ("Naive", "Exhausted"),
    ("Activated", "Exhausted"),
]

results_pairs = []
rows_load_pairs = []

for c1, c2 in pairs:
    df_pair = df_good_cells[df_good_cells["category"].isin([c1, c2])].copy()
    X_pair = df_pair[feature_cols].values
    y_pair = df_pair["category"].values

    imputer_pair = SimpleImputer(strategy="mean")
    X_pair_imp = imputer_pair.fit_transform(X_pair)

    scaler_pair = StandardScaler()
    X_pair_scaled = scaler_pair.fit_transform(X_pair_imp)
```

2.6.4 Modelli per la classificazione delle categorie

Per riconoscere automaticamente le tre categorie funzionali di linfociti (Naive, Activated, Exhausted) è stata utilizzata come base unicamente la popolazione di cellule con segmentazione buona, contenuta nel dataframe *df_good_cells*, così da ridurre il rumore legato a contorni errati e allenare i modelli su esempi morfologicamente affidabili. Da questo dataframe sono stati costruiti la matrice delle feature X (le stesse 23 feature morfologiche e fotometriche già definite per il modello good/bad, ad esempio area, perimeter, circularity, solidity, intensità e contrasto) e il vettore delle etichette di categoria category, quindi suddivisi in training e test tramite *train_test split* con test size pari al 20% e stratificazione sulle classi, per preservare le proporzioni tra Naive, Activated ed Exhausted.

Entrambi i classificatori sono stati implementati come pipeline scikit-learn, composte da imputazione dei valori mancanti con la mediana (*SimpleImputer(strategy="median")*, standardizzazione delle feature (*StandardScaler*) e modello supervisionato multiclasse. Per la Logistic Regression è stato adottato un modello multinomiale con regolarizzazione L2 e *class_weight="balanced"* per compensare l'eventuale squilibrio tra categorie (*Logistic Regression(multiclass = "multinomial", max_iter = 2000, class_weight = "balanced")*), mentre per la Random Forest è stato utilizzato un ensemble di 200 alberi con profondità massima libera e foglie minime pari a 2 (*RandomForestClassifier (n_estimators = 200, max_depth = None, min_samples_leaf=2, random_state = 42, n_jobs=-1*).

Per sfruttare la complementarità tra modello lineare e non lineare è stato inoltre costruito un semplice ensemble a livello di probabilità: per ogni cellula del test sono state calcolate le matrici di probabilità per classe prodotte dai due modelli (*predict_proba*), quindi combinate come media pesata $0.5\, p_{\mathrm{LR}} + 0.5\, p_{\mathrm{RF}}$; la categoria finale è stata assegnata scegliendo, per ogni riga, l'indice di probabilità massima e traducendolo nel corrispondente nome di classe, usando l'ordinamento *classes_* della Logistic Regression. Anche per l'ensemble è stato calcolato il classification report.

```python
# Pipeline 1: Multinomial Logistic Regression
pipeline_cat_logreg = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        (
            "logreg_multiclass",
            LogisticRegression(
                multi_class="multinomial",
                max_iter=2000,
                class_weight="balanced",
            ),
        ),
    ]
)
```

```python
# Pipeline 2: Random Forest multiclass
pipeline_cat_rf = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        (
            "rf_multiclass",
            RandomForestClassifier(
                n_estimators=200,
                max_depth=None,
                min_samples_leaf=2,
                random_state=42,
                n_jobs=-1,
                class_weight=None,
            ),
        ),
    ]
)
```

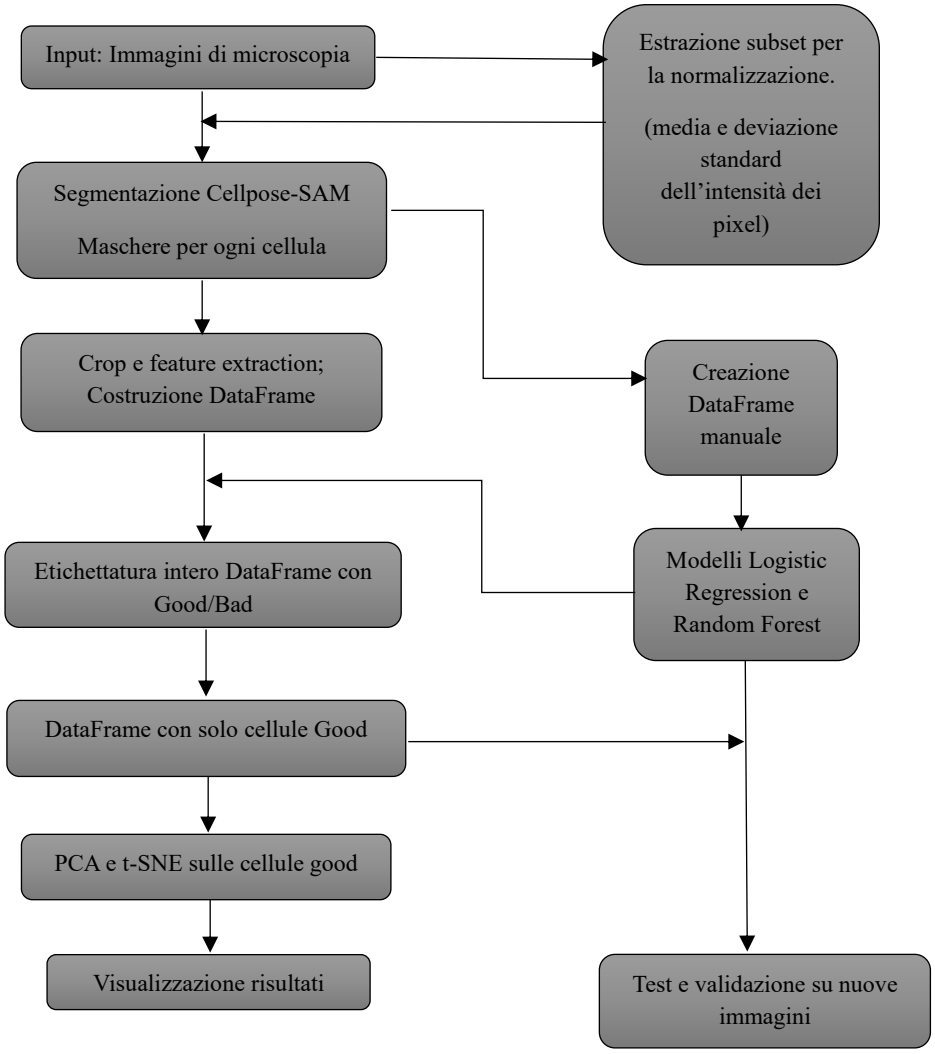
```python
# Train both models
pipeline_cat_logreg.fit(feature_train, label_train)
pipeline_cat_rf.fit(feature_train, label_train)
```

```python
# Ensemble probabilities on test set
proba_logreg_test_cat = pipeline_cat_logreg.predict_proba(feature_test)
proba_rf_test_cat = pipeline_cat_rf.predict_proba(feature_test)

proba_ensemble_test_cat = 0.5 * proba_logreg_test_cat + 0.5 * proba_rf_test_cat
pred_indices_ensemble = proba_ensemble_test_cat.argmax(axis=1)

class_labels = pipeline_cat_logreg.classes_
pred_test_ensemble_labels = class_labels[pred_indices_ensemble]
```

Capitolo 3 Risultati

3.1 Descrizione complessiva della pipeline

Questo paragrafo riassume in forma di schema a blocchi la pipeline completa sviluppata per l'analisi delle immagini di linfociti, dalla microscopia grezza fino alla classificazione automatica delle cellule. Il diagramma evidenzia l'ordine dei moduli principali – normalizzazione delle immagini, segmentazione Cellpose-SAM, estrazione dei crop e delle feature, addestramento dei modelli good/bad, selezione delle sole cellule buone e successiva classificazione nelle categorie funzionali – insieme ai flussi di dati che li collegano.



3.2 Risultati di segmentazione

In questa sezione si riportano i risultati ottenuti dalla segmentazione automatica delle immagini di microscopia mediante il modello Cellpose-SAM. Visivamente, le maschere prodotte dal modello seguono in maniera soddisfacente il contorno cellulare, riuscendo a separare la maggior parte delle cellule adiacenti e a preservare la forma complessiva dei linfociti. Eventuali errori residui si manifestano principalmente sotto forma di cellule fuse in regioni ad alta densità o di piccoli frammenti in corrispondenza di artefatti o rumore.

Non disponendo di una ground truth completamente annotata per tutte le immagini, la qualità della segmentazione è stata valutata in modo semi-quantitativo: un insieme di immagini è stato ispezionato dall'autore del lavoro e le cellule sono state successivamente etichettate come "good" o "bad" in base alla correttezza del contorno, fornendo il dataset di riferimento per l'addestramento del classificatore di qualità. In questo modo, la pipeline non richiede che Cellpose-SAM produca maschere perfette per tutte le cellule, ma sfrutta un modello supervisionato a valle per filtrare le segmentazioni inaffidabili e utilizzare solo le cellule "good" nelle analisi morfologiche e nella classificazione delle categorie funzionali.

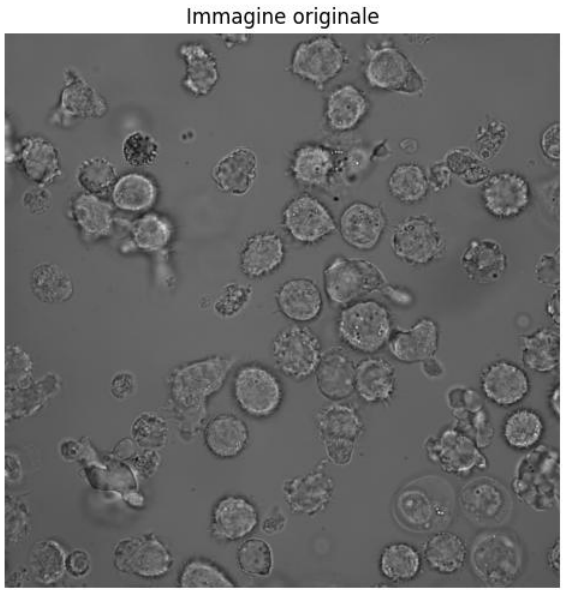


Figura 3.1

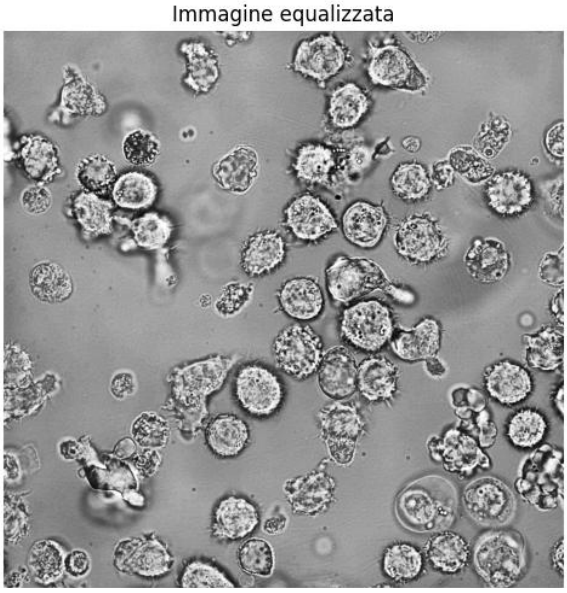


Figura 3.2

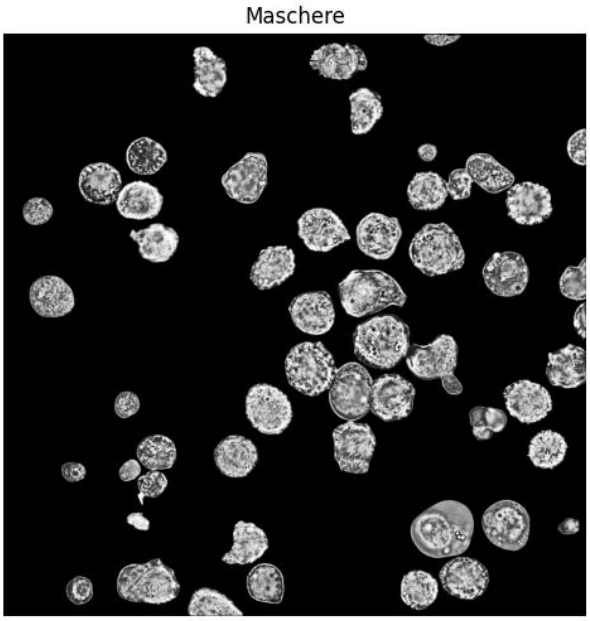


Figura 3.3

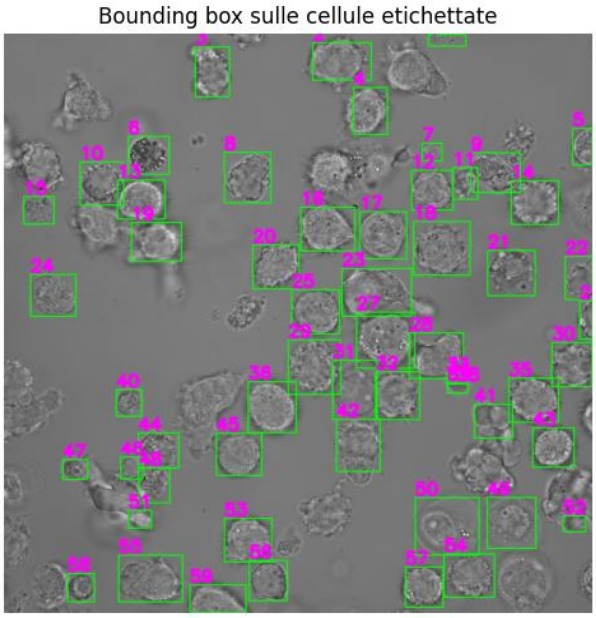


Figura 3.4

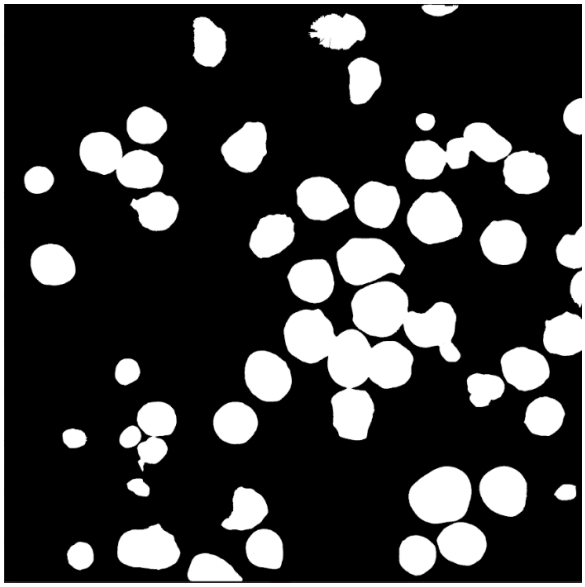


Figura 3.5 Maschere salvate

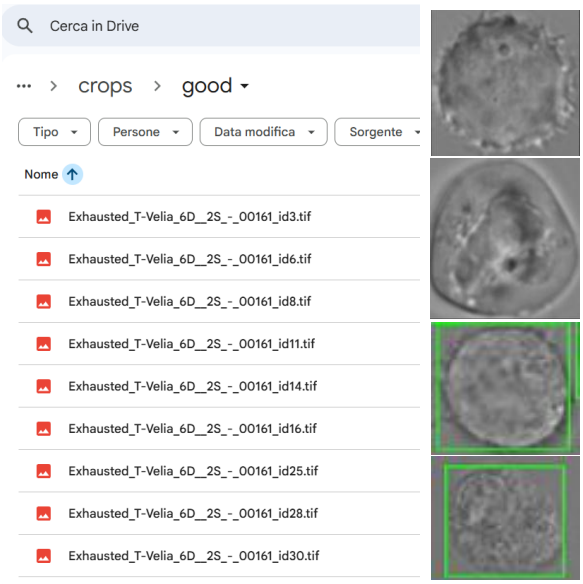


Figura 3.6 Cartella di salvataggio dei crops "good" ed esempi di crops good

3.3 Risultati di classificazione

In questa sezione vengono presentate le prestazioni dei modelli di classificazione sviluppati nella pipeline, con un doppio obiettivo:

- Distinguere automaticamente tra cellule affidabili (*good*) e non (*bad*) ai fini di analisi.
- Assegnare alle sole cellule buone una delle tre categorie funzionali (Naive, Activated, Exhausted).

Per entrambe le fasi sono stati valutati diversi modelli supervisionati, basati su Logistic Regression e Random Forest, e ne sono state analizzate le metriche principali (accuracy, precision, recall e f1-score) sul set di test mantenuto separato dai dati di addestramento.

3.3.1 Classificazione good / bad

La prima fase di classificazione ha riguardato la distinzione tra cellule con segmentazione affidabile (*good*) e non affidabile (*bad*), utilizzando come ground truth le etichette manuali assegnate dall'operatore al sottoinsieme annotato. Su questo dataset sono stati addestrati e confrontati due modelli supervisionati binari, Logistic Regression e Random Forest, entrambi implementati come pipeline e successivamente valutati sul test set tenuto separato.

Tabella 3.1 Classification Report Logistic

	precision	recall	f1-score	support
bad	0,76	0,96	0,85	50
good	0,94	0,68	0,79	47
accuracy			0,82	97
macro avg	0,85	0,82	0,82	97
weighted avg	0,85	0,82	0,82	97

Tabella 3.2 Classification Report Random Forest

	precision	recall	f1-score	support
bad	0,73	0,98	0,84	50
good	0,97	0,62	0,75	47
accuracy			0,8	97
macro avg	0,85	0,8	0,8	97
weighted avg	0,85	0,8	0,8	97

Le metriche di classificazione riportate in Tabella 3.1 / 3.2 mostrano che i due modelli raggiungono prestazioni molto simili in termini di accuratezza complessiva (circa 0.82 per Logistic Regression e 0.80 per Random Forest), con valori di f1-score macro intorno a 0.80 per entrambe le classi good e bad (dove *precision* misura la purezza delle predizioni positive, *recall* la frazione di veri positivi correttamente individuati e l'*f1-score* la loro media armonica, utile quando le classi sono sbilanciate). In particolare, la Logistic Regression tende a privilegiare leggermente la classe good in termini di f1-score, mentre la Random Forest massimizza la sensibilità sulla classe bad, riducendo il numero di cellule segmentate male che passano il filtro di qualità. Le learning curve sottostanti evidenziano che, per tutti e due i modelli, le curve di training e validation si avvicinano progressivamente all'aumentare dei campioni di training, indicando un buon compromesso tra bias e varianza e una quasi assenza di overfitting marcato nel range di dati disponibile.

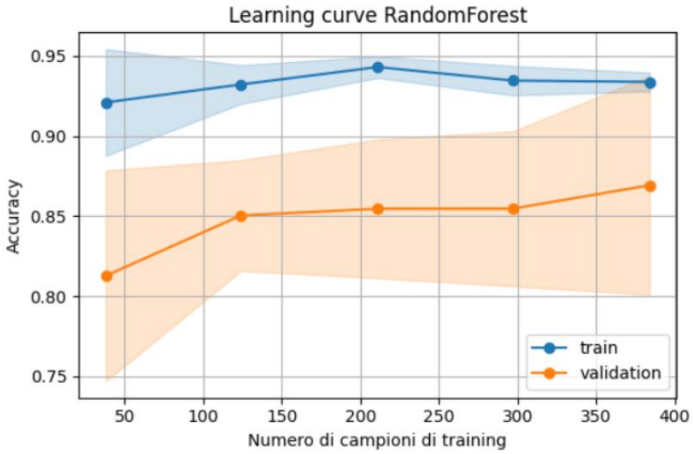


Figura 3.7

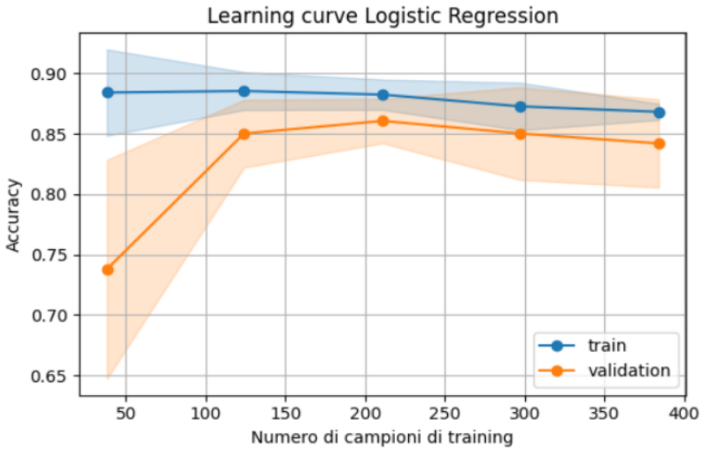


Figura 3.8

Per interpretare quali parametri morfologici e fotometrici contribuiscono maggiormente alla decisione good/bad è stata infine calcolata l'importanza delle feature tramite i coefficienti della Logistic Regression e le *feature importance* della Random Forest, combinando i due ranking in un unico indice (File "Importance_feature_LR_RF.xlsx", dove un valore più alto indica una maggiore capacità della feature di discriminare tra le due classi nel modello lineare e in quello ad alberi). Le feature con peso più elevato risultano l'eccentricità, la rotondezza e l'aspect ratio, seguite da area e minor axis, confermando che il modello di qualità si basa prevalentemente sulla forma del contorno e sulla compattezza della cellula, come atteso dal punto di vista morfologico.

.....

Tabella 3.3 Importance Feature, Logistic Regression - Random Forest

feature	log_coef	log_abs_coef	rf_importance	combined_rank
eccentricity	-1,153	1,153	0,211	2
circularity	1,129	1,129	0,178	5
aspect_ratio	-0,692	0,692	0,178	7
area	-0,763	0,763	0,061	9
minor_axis	1,125	1,125	0,059	9
solidity	0,545	0,545	0,094	11
perimeter	0,481	0,481	0,038	16
extent	0,293	0,293	0,043	17
major_axis	0,257	0,257	0,034	20
ratio_mean	0,559	0,559	0,009	20
visibility	-0,179	0,179	0,011	23
min_int	0,466	0,466	0,009	24
mean_int	-0,173	0,173	0,01	25
std_int	0,079	0,079	0,013	30
contrast_mean	-0,151	0,151	0,009	31
contrast_max	-0,107	0,107	0,01	32
edge_int_mean	0,109	0,109	0,008	34
bg_median	-0,164	0,164	0,006	34
median_int	-0,114	0,114	0,006	35
bg_mean	0,109	0,109	0,006	35
max_int	-0,015	0,015	0,002	43
quality_score	-0,013	0,013	0,006	43

3.3.2 Classificazione Naive vs Activated vs Exhausted

Per la classificazione funzionale sono state considerate esclusivamente le cellule con segmentazione buona presenti nel dataframe *df_good_cells*, etichettate come Naive, Activated o Exhausted. Su questo dataset sono stati addestrati due modelli multiclasse, Logistic Regression e Random Forest, entrambi incapsulati in pipeline con imputazione dei valori mancanti e standardizzazione delle feature, utilizzando una suddivisione train/test stratificata per mantenere il bilanciamento tra le tre categorie.

	precision	recall	f1-score	support
Activated	0,99	0,81	0,89	90
Exhausted	0,95	0,99	0,97	348
Naive	0,97	0,97	0,97	78
accuracy			0,96	516
macro avg	0,97	0,93	0,94	516
weighted avg	0,96	0,96	0,96	516

Tabella 3.4 Random Forest Classification Report 3 categorie

	precision	recall	f1-score	support
Activated	0,6	0,86	0,71	90
Exhausted	0,95	0,82	0,88	348
Naive	0,86	0,96	0,91	78
accuracy			0,85	516
macro avg	0,81	0,88	0,83	516
weighted avg	0,88	0,85	0,86	516

Tabella 3.5 Logistic Regression Classification Report 3 categorie

Le metriche di classificazione riportate in Tabella 3.4 / 3.5 mostrano che entrambi i modelli sono in grado di riconoscere le tre classi con buona accuratezza complessiva, con valori di *precision*, *recall* e *f1-score* per ciascuna categoria compresi, rispettivamente, in un intervallo circa 0.8–0.95, 0.75–0.9 e 0.8–0.9 (dove *precision* misura la purezza delle predizioni di una classe, *recall* la frazione di elementi di quella classe correttamente individuati e l'*f1-score* la loro media armonica). La Random Forest tende a catturare meglio le non linearità tra feature morfologiche e fotometriche, ottenendo un lieve incremento di f1-score medio

rispetto alla Logistic Regression, mentre quest'ultima mantiene una struttura decisionale più interpretabile grazie alla forma lineare degli iperpiani di separazione nello spazio delle feature. Le matrici di confusione a intensità di colore sottostanti confermano visualmente questo comportamento: nel caso della Random Forest la maggior parte delle osservazioni si concentra sulla diagonale, con pochissime confusioni tra Activated ed Exhausted, mentre per la Logistic Regression gli errori si distribuiscono prevalentemente tra queste due classi, lasciando comunque ben riconosciuta la popolazione Naive.

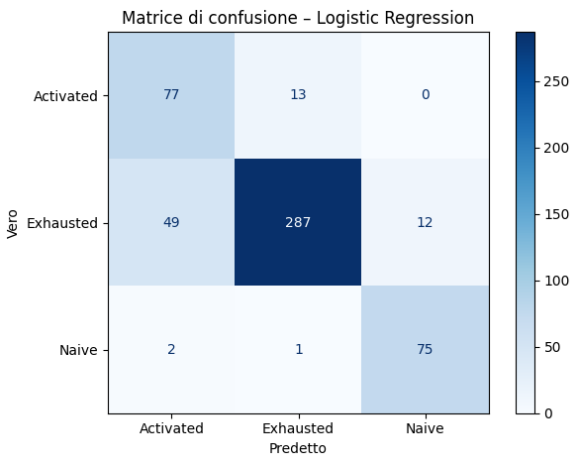


Figura 3.9

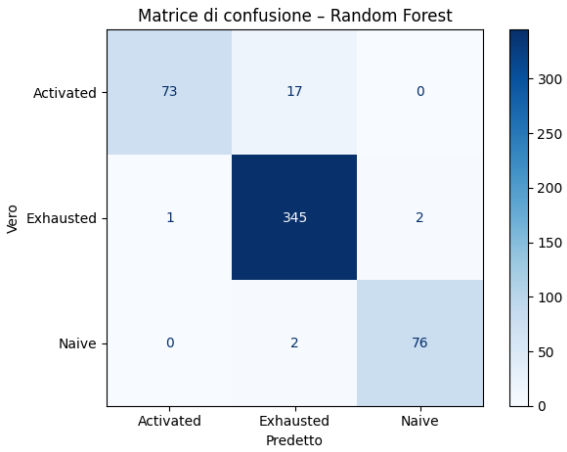


Figura 3.10

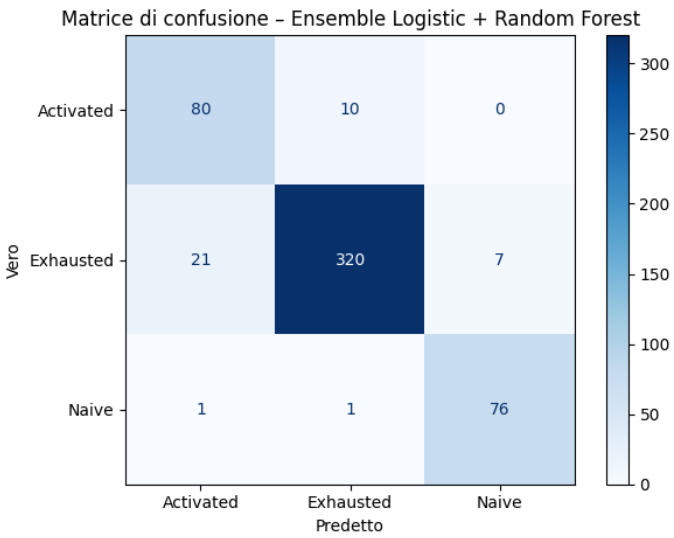


Figura 3.11

Le mappe PCA e t-SNE riportate in Figura 3.11 / 3.12 forniscono un supporto visivo alla bontà della classificazione: nei primi due/tre componenti principali, che spiegano complessivamente oltre l'80% della varianza, la classe Naive occupa una regione ben separata dalle classi Activated ed Exhausted, mentre queste ultime risultano parzialmente sovrapposte ma presentano differenze nella distribuzione lungo PC1–PC2 e nelle coordinate t-SNE. I loadings delle prime componenti (Tab. 3.6") indicano che la separazione tra categorie è governata principalmente da feature di forma (aspect ratio, eccentricità, area) e di contrasto interno/di bordo, coerentemente con l'ipotesi che stati funzionali diversi dei linfociti presentino pattern morfologici e fotometrici distinti.

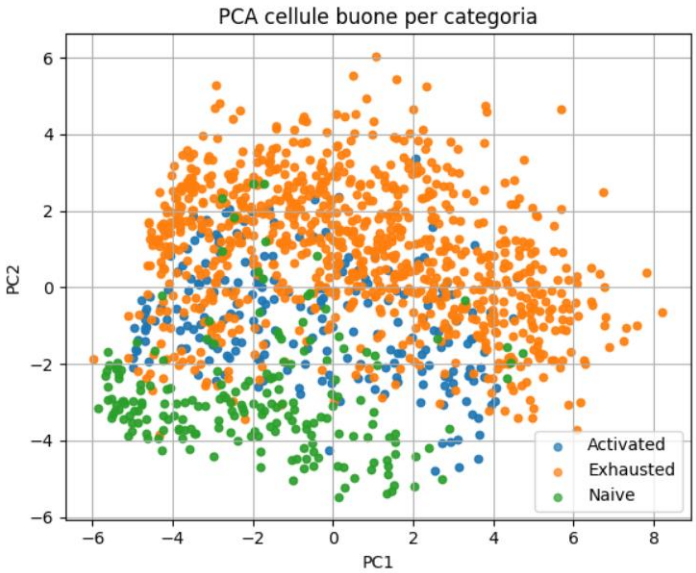


Figura 3.12

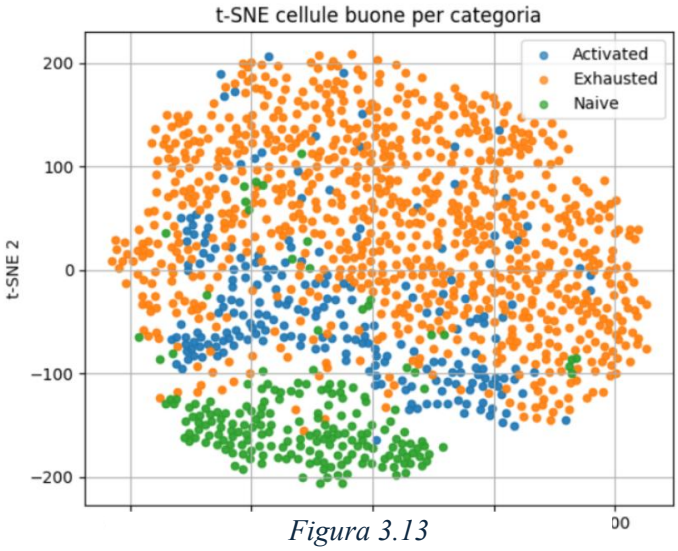
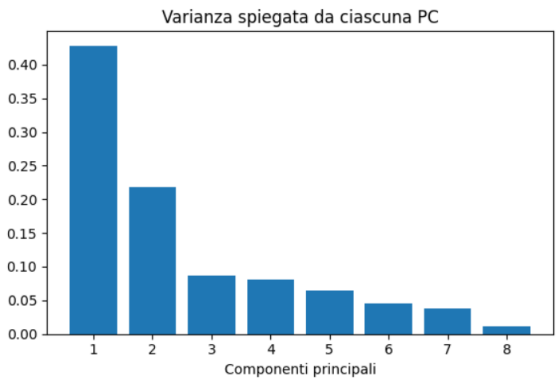
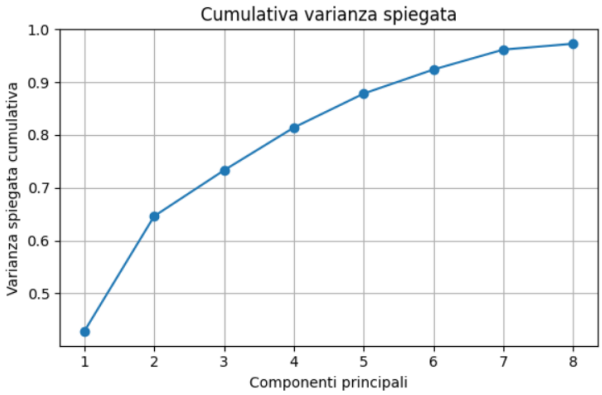


Figura 3.13





PC	feature	loading	abs_loading
PC1	contrast_max	0,318	0,318
	visibility	0,315	0,315
	contrast_mean	0,315	0,315
	quality_score	0,304	0,304
	edge_int_mean	-0,3	0,3
	bg_mean	-0,3	0,3
	bg_median	-0,298	0,298
	ratio_mean	0,292	0,292
	median_int	0,289	0,289
	mean_int	0,272	0,272
PC2	major_axis	0,395	0,395
	perimeter	0,393	0,393
	area	0,384	0,384
	minor_axis	0,379	0,379
	max_int	0,268	0,268
	min_int	-0,252	0,252
	circularity	-0,238	0,238
	std_int	0,23	0,23
	aspect_ratio	0,147	0,147
	eccentricity	0,146	0,146
PC3	aspect_ratio	0,669	0,669
	eccentricity	0,667	0,667
	minor_axis	-0,188	0,188
	area	-0,131	0,131
	perimeter	-0,115	0,115
	min_int	0,111	0,111
	mean_int	0,093	0,093
	max_int	0,087	0,087
	median_int	0,073	0,073
	major_axis	-0,047	0,047

Tabella 3.6 Top 10 Loadings delle 3 PC

3.3.3 Classificazione tra Linfociti T Attivi vs Exhausted

Lo studio delle differenze morfologiche e fotometriche tra categorie funzionali è stato condotto su tutte le coppie di classi (Naive vs Activated, Naive vs Exhausted, Activated vs Exhausted), limitandosi in ogni caso alle sole cellule con segmentazione buona. Tuttavia, l'attenzione principale è rivolta al confronto tra linfociti T Activated ed Exhausted, che rappresenta il nucleo applicativo del lavoro in quanto più direttamente rilevante per la caratterizzazione dello stato di attivazione cronica dei T-cell.

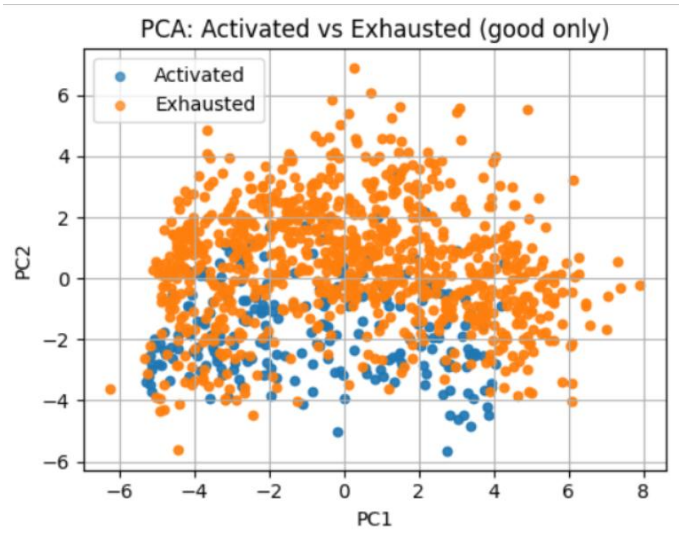


Figura 3.14

Nel sottospazio definito dai primi componenti principali, Activated ed Exhausted mostrano una separazione solo parziale: i primi due componenti spiegano circa il 60% della varianza totale e rivelano una tendenza degli Exhausted a occupare regioni con valori mediamente maggiori di PC1 e PC2, mentre gli Activated risultano più concentrati in un'area a PC2 più bassi. Il contributo dei singoli parametri alle componenti principali evidenzia che la discriminazione tra le due classi è governata soprattutto da feature di contrasto e intensità (visibility, contrast_mean, contrast_max, bg_mean, edge_int_mean per PC1) e da feature di dimensione e geometria (perimeter, major_axis, area, minor_axis e min_int per PC2), suggerendo che lo stato esausto si associa a cellule tendenzialmente più grandi e con profili di intensità più marcati rispetto agli Activated.

La mappa t-SNE costruita sulle stesse feature per la sola coppia Activated–Exhausted mostra due distribuzioni parzialmente sovrapposte ma con regioni a maggiore densità specifiche per ciascuna classe, in accordo con la capacità dei modelli di classificazione di distinguere con buona accuratezza tra i due stati funzionali. In termini applicativi, queste analisi confermano che le differenze morfologiche e fotometriche catturate automaticamente dalla pipeline contengono informazione sufficiente per discriminare tra linfociti Activated ed Exhausted, fornendo un supporto quantitativo all'interpretazione biologica dei risultati di classificazione.

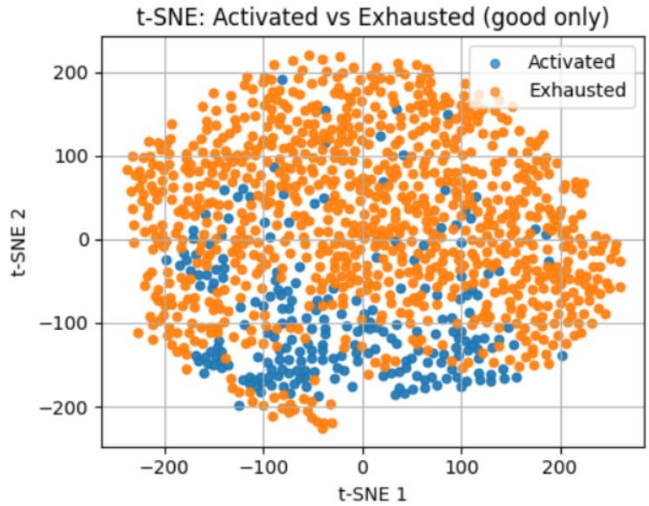


Figura 3.15

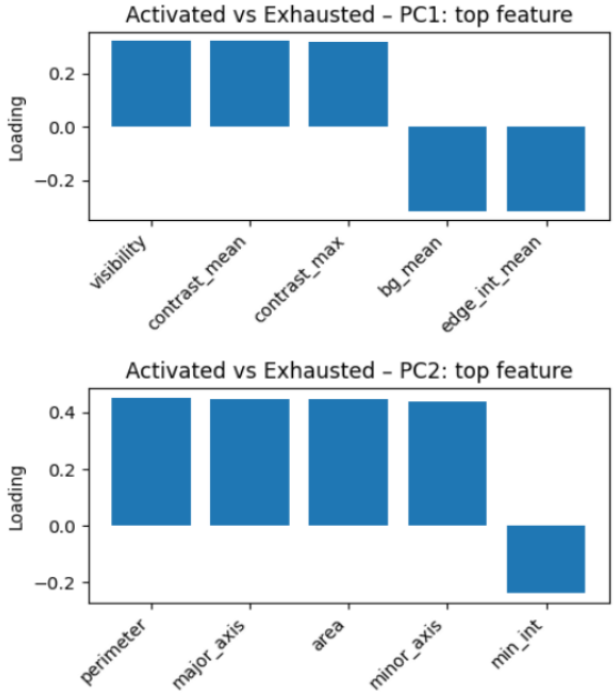


Figura 3.16

Capitolo 4 Conclusioni

4.1 Validità del workflow sui linfociti T

Per valutare la capacità di generalizzazione del workflow proposto, i modelli di segmentazione e classificazione sono stati applicati a due dataset indipendenti rispetto a quello utilizzato per l'addestramento. Il primo, denominato "*1Splus*", contiene immagini di linfociti T inizialmente attivati e successivamente riattivati dopo 6 giorni; da un punto di vista biologico ci si attende che una parte delle cellule mostri caratteristiche intermedie tra lo stato Activated e lo stato Exhausted, fornendo quindi un banco di prova particolarmente sfidante per la pipeline. Il secondo dataset, indicato come "*D1"*, è composto da linfociti T attivati solo per mezza giornata, per i quali è ragionevole aspettarsi un fenotipo ancora vicino alla condizione Naive.

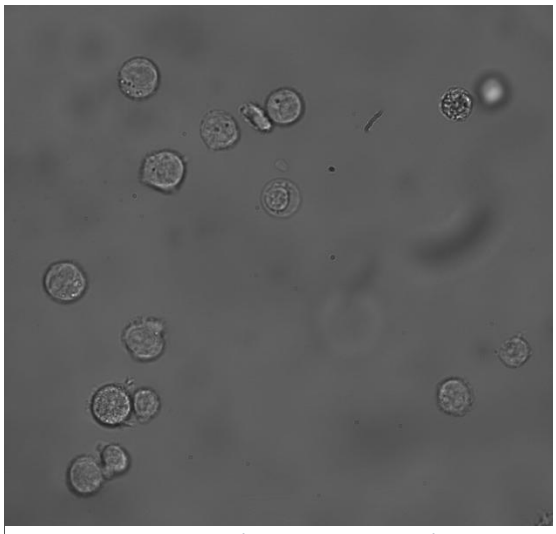


Figura 4.1 Immagine D1

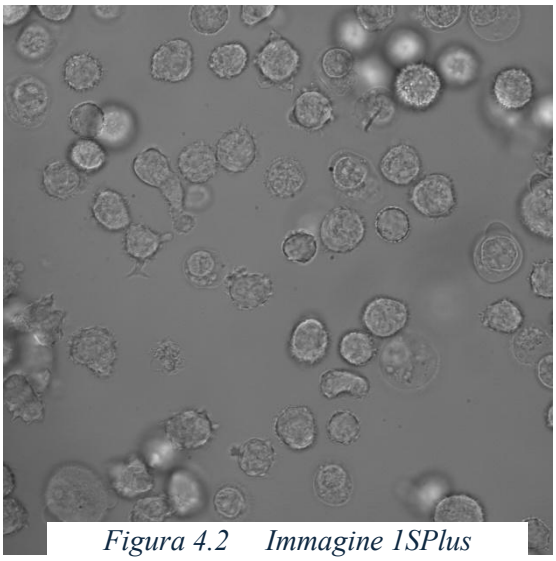


Figura 4.2 Immagine 1SPlus

Su entrambi i dataset il flusso di analisi è stato mantenuto identico a quello descritto nel Capitolo 3: segmentazione automatica delle immagini, calcolo delle feature morfologiche e fotometriche per ogni cellula, classificazione della qualità della segmentazione (modelli good/bad) e, per le sole cellule giudicate good, assegnazione della categoria funzionale tramite i modelli multiclasse Logistic Regression, Random Forest ed ensemble. Le distribuzioni finali delle classi predette risultano coerenti con le aspettative biologiche: nel dataset D1 la maggior parte delle cellule viene

etichettata come Naive, mentre in 1Splus prevalgono le classi Activated ed Exhausted, con una frazione non trascurabile di cellule classificate come Exhausted che riflette la storia di riattivazione prolungata. Questi risultati indicano che la pipeline mantiene buone prestazioni anche su immagini non viste e che i pattern morfologici appresi sui dati di training sono in grado di descrivere in modo plausibile gli stati funzionali di nuovi linfociti T acquisiti in condizioni sperimentali differenti.

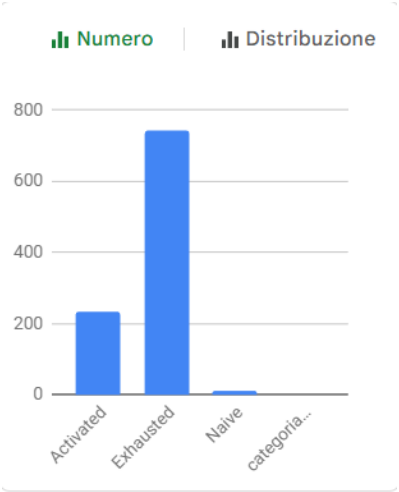


Figura 4.3 Cellule classificate in 1Splus

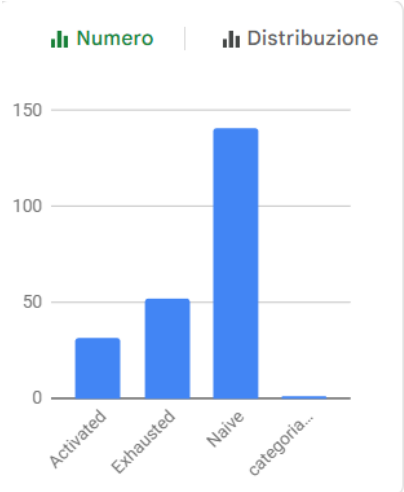


Figura 4.4 Cellule classificate in D1

Più alta | Più bassa

VALORE	FREQUENZA
Exhausted	741
Activated	232
Naive	12
categoria_predetta_...	1

Figura 4.5 1Splus

Più alta | Più bassa

VALORE	FREQUENZA
Naive	141
Exhausted	52
Activated	32
categoria_predetta_...	1

Figura 4.6 D1

In termini numerici, dunque, abbiamo ottenuto: Nel dataset D1 circa il *63%* delle cellule viene classificato come *Naive*, il *23%* come *Exhausted* e il *14%* come *Activated*, mentre in 1Splus circa il *75%* risulta *Exhausted*, il *24% Activated* e *solo l'1% Naive*.

4.2 Limiti del dataset e della microscopia Brightfield

Dal punto di vista dei dati, il principale limite è la dimensione relativamente contenuta e lo sbilanciamento tra classi, con una sovra-rappresentazione di alcune condizioni biologiche rispetto ad altre (ad esempio un numero molto maggiore di cellule Exhausted rispetto alle Activated o Naive in alcuni esperimenti). Questo squilibrio può indurre i modelli a privilegiare le classi più frequenti, richiedendo l'uso di strategie come la pesatura delle classi o la scelta di metriche bilanciate per una valutazione corretta delle prestazioni. Inoltre, le immagini provengono da un solo setup sperimentale e da un numero limitato di donatori/repliche, il che può introdurre bias legati alle condizioni specifiche di acquisizione e ridurre la capacità di generalizzazione verso altri laboratori o protocolli.

La microscopia brightfield presenta a sua volta alcune limitazioni note: il contrasto intrinseco delle cellule non marcate è spesso basso, i bordi possono risultare poco definiti e l'informazione di fase (spessore ottico) è compressa in un'unica immagine di intensità. Questo rende più difficile la segmentazione automatica rispetto a tecniche a contrasto di fase o fluorescenza e può introdurre incertezza nella misura di parametri morfologici fini, soprattutto in presenza di cellule ad alta densità o leggermente fuori fuoco. Inoltre, la brightfield non fornisce direttamente informazioni molecolari o funzionali (espressione di marcatori specifici), per cui l'intera classificazione si basa su feature puramente morfologiche e di intensità, che possono non catturare tutte le sfumature biologiche dello stato dei linfociti T.

4.3 Possibili estensioni: altri tipi cellulari e modalità di imaging

Sebbene il workflow sia stato sviluppato e validato specificamente su linfociti T in microscopia brightfield, la struttura modulare della pipeline (segmentazione, estrazione feature, classificazione good/bad, classificazione funzionale) la rende concettualmente applicabile ad altri tipi cellulari e a diverse modalità di imaging. A fronte di un opportuno *re-training* dei modelli con annotate specifiche, lo stesso codice può essere riutilizzato per cellule con morfologie differenti (ad esempio cellule

tumorali, cellule epiteliali o staminali), modificando solo i parametri di segmentazione e, se necessario, l'insieme di feature morfologiche e fotometriche più informative per il nuovo contesto.

Un'estensione naturale riguarda anche l'impiego di altre tecniche microscopiche, come fluorescenza, contrasto di fase o imaging quantitativo di fase, che offrono un contrasto più elevato e canali addizionali rispetto alla brightfield; in tali casi la pipeline potrebbe integrare feature derivate da più canali o mappe di fase e combinare informazione morfologica e molecolare, potenzialmente migliorando la discriminazione tra stati funzionali o sottopopolazioni cellulari. L'adozione dello stesso schema di analisi su dataset multi-modalità permetterebbe inoltre di confrontare in modo sistematico quanto le diverse tecniche di imaging contribuiscono alla separabilità delle classi, mantenendo invariata l'architettura software ma adattando solo i modelli di segmentazione e classificazione.

4.4 Contributi e prospettive applicative

Il lavoro svolto ha portato alla definizione di una pipeline completa per l'analisi morfologica di linfociti T in microscopia brightfield, che integra in modo coerente segmentazione automatica, estrazione di feature, valutazione della qualità della segmentazione e classificazione funzionale in Naive, Activated ed Exhausted. Il principale contributo metodologico consiste nell'avere esplicitato la qualità del contorno come variabile *good/bad* modellata da classificatori supervisionati, così da filtrare le cellule segmentate in modo non affidabile prima della fase di analisi biologica. A ciò si aggiunge la dimostrazione che un insieme relativamente semplice di parametri morfologici e fotometrici, combinati tramite modelli di machine learning interpretabili (Logistic Regression, Random Forest, ensemble), è sufficiente per discriminare con buona accuratezza gli stati funzionali dei T-cell, in linea con recenti studi che utilizzano la morfologia come proxy della funzione immunitaria.

Dal punto di vista degli sviluppi futuri, la pipeline potrà essere estesa sia in profondità sia in ampiezza. In profondità, sarà possibile esplorare modelli più complessi (ad

esempio reti neurali profonde o metodi self-supervised) mantenendo però l'attenzione sull'interpretabilità, ad esempio integrando tecniche di *feature attribution* o confrontando sistematicamente i pattern morfologici appresi con quelli descritti dalla letteratura biologica. In ampiezza, il workflow potrà essere riadattato ad altri tipi cellulari e ad altre modalità di imaging (microscopia fluorescente, quantitativa di fase, imaging multiplex), integrando feature di natura diversa in un'unica rappresentazione morfo-funzionale e collegando l'output del classificatore con dati omici o funzionali.

Le potenziali applicazioni cliniche e sperimentali di un approccio di questo tipo sono molteplici. In ambito preclinico, una pipeline automatizzata basata su immagini potrebbe supportare studi ad alto contenuto (high-content screening) per valutare rapidamente l'effetto di farmaci o condizioni di stimolazione sullo stato dei linfociti T, riducendo la dipendenza esclusiva da marcatori molecolari e da citometria tradizionale. In prospettiva clinica, l'uso combinato di morfologia quantitativa e machine learning potrebbe contribuire a definire saggi rapidi per monitorare lo stato funzionale dei T-cell di un paziente, potenzialmente utili per la selezione o il follow-up di terapie immunitarie, in accordo con la crescente attenzione verso il *profilo morfologico* come strumento complementare alle tecniche multi-omiche.

Bibliografia

Figura 1.1: The light path of a darkfield microscope. From Hart and Shears, 1997

Figura 2.3: Google Colab logo

Librerie di calcolo e data analysis

- *Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. Nature, 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2*
- *Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17, 261–272.*
- *McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), Proceedings of the 9th Python in Science Conference (SciPy 2010) (pp. 51–56). [pandas].*

Machine learning e image processing

- *Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.*
- *Van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., et al. (2014). scikit-image: Image processing in Python. PeerJ, 2, e453.*
- *Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.*

Visualizzazione

- *Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.*
- *Waskom, M. L. (2021). seaborn: Statistical data visualization. Journal of Open Source Software, 6(60), 3021. https://doi.org/10.21105/joss.03021*

Ringraziamenti

Ai miei genitori,

Grazie, davvero, per non avermi mai fatto mancare nulla. E, ancora più importante, per aver fatto capire al me bambino — quello che magari piangeva per un paio di scarpe (un po' moccioso e capriccioso) — che in realtà non gli è mai mancato niente.

Grazie per la fiducia che avete sempre riposto in me: sapere di poter cadere e rialzarmi liberamente mi ha insegnato a non temere le cadute e ad amare le risalite.

Vi ringrazio anche per il sostegno che mi avete dato lungo questo percorso, spesso silenzioso (diciamo che non sono il figlio più "inclusivo" quando si parla di stare sotto pressione, ihi), ma ho imparato a capire quanto avete fatto e continuerete a fare il tifo per me. Di questo vi sono infinitamente grato. Grazie.

A Ila, la mia sorellina (anche se abbiamo la stessa età).

Lo sai che ancora non so rispondere alla domanda "Com'è avere un gemello?" Forse perché, in realtà, non c'è una risposta semplice, tipo "è come avere un fratello" o "è come avere un amico". A volte siamo fratelli, a volte amici, semplici coinquilini e, in passato, persino un po' sconosciuti — se non addirittura nemici — quando eravamo più piccoli e i grandi non riuscivano a scindere le nostre date di nascita e a distinguerci come due individui diversi.

Devo a te la parentesi del passato: grazie per essere stata la prima tra noi due a rompere quel muro invisibile, e per avermi fatto capire che mettersi sul letto dell'altro per parlare di un problema o anche solo per chiacchierare non era strano, ma anzi, bellissimo.

È stato altrettanto bello scoprire, col tempo, come tu abbia una vita tutta tua e che, ogni tanto, mi lasci entrare per chiedere un consiglio, un po' di supporto o semplicemente conforto.

P.S. Prometto che proverò a farlo di più anch'io.

Grazie per essere stata la persona che più ha tifato per me e che ha sempre saputo guardare oltre le mie finte rassicurazioni: mi cagavo sotto, e tu lo capivi subito.

Senza di te sarebbe stato tutto molto più difficile. Ti voglio tanto bene.

A tutti i cugini,

Davide in particolare con cui ho sempre condiviso un po' di più la mia vita e con il quale ci siamo trovati più o meno ad affrontare problemi non molto diversi tra loro, ti sono grato dei consigli che mi hai dato negli anni e mi auguro, perché no, di aver ricambiato e poter continuare a farlo, magari condividendo assieme qualche altra esperienza.

A Keka,

beh che dire... non saprei chi ringraziare per questo quindi ringrazio direttamente te: grazie per esserci incontrati, ormai più di tre anni fa. ne ho vissuti molti di più, ma posso giurare che questi ultimi tre sono stati i più intensi di sempre. sei una delle persone più forti che io abbia mai conosciuto, anche se a distanza di settimane posso immaginarti in lacrime mentre ascolti questi ringraziamenti, cosa che, in fondo, è un'altra dimostrazione della tua forza, che non ha paura di mostrare le proprie emozioni.

Non so quante di queste riconoscerai ma in questi tre anni mi hai insegnato tante cose (anche se non sono stato uno studente modello), molti le darebbero per scontate ma con una maestra come te, non dare per scontate le cose è nelle prime lezioni.

Mi hai insegnato ad amare innanzitutto, un qualcosa che non arriva con un manuale d'istruzioni e che se posta in modo sbagliato può fare molto male. Mi hai insegnato che l'amicizia va custodita e onorata quanto un amore. Mi hai insegnato a stare insieme ad un'altra persona, con tutto ciò che ne comporta, e che crescere insieme è decisamente più divertente e meno spaventoso. Mi hai insegnato ad essere più spensierato (nei miei limiti almeno), a non stressarmi troppo e, specialmente ad accettarmi per come sono, cosa che probabilmente hai fatto prima tu di me, facendomi capire che quella maledetta perfezione, che, a parole disprezzo, ma che a fatti rincorro, non deve logorarmi. Mi hai fatto capire che sono proprio le imperfezioni a rendermi ciò che sono e che sarò.

Sei un esempio di maturità, bontà, gentilezza e solarità (per citarne qualcuna), che porterò sempre con me. A prescindere da ciò che il destino ci riserverà, spero che in un modo o nell'altro entrambi riusciremo a far parte della vita dell'altro e magari riuscire a dedicarti altri pensieri come questo. Grazie per avermi sostenuto dal primo momento, Ti auguro la felicità e che il mondo si accorga di quanto lo migliori.

T vogl ben assai.

All'aula studio

Ragazzi, grazie. È sempre stato difficile spiegare a chi non ne fa parte cosa abbiamo costruito in questi anni; tant'è che mi sono sempre limitato a dire "vado in uni", sapendo quanto fosse riduttivo. In realtà andavo in una seconda casa, con persone che stavano vivendo — o, più spesso nel mio caso, avevano già vissuto — periodi simili a quelli che mi

aspettavano, e su cui potevo contare per una mano, una distrazione o semplicemente una chiacchiera.

Ringrazio quindi Ivo, Mattia, Davidone, Duilio, Peppe e Ale: mi avete sempre accolto con un sorriso, senza mai farvi problemi a "perdere" due minuti in conversazione.

Grazie a Luca A., Luca B., Lucianina e Merluzzo per la gentilezza e la bontà che riuscite a trasmettermi anche quando penso di non meritarle.

Grazie a Matteo, Amedeo e Ale per gli scleri delle sei del pomeriggio e per aver trovato in voi dei veri, buoni amici.

Grazie a chi non si è mai tirato indietro e non mi ha mai negato un aiuto, al punto da trasformare i miei esami in veri progetti di gruppo. Di questo ringrazio Raffo, Tommi, Bro e Sasà (ma in realtà, grazie a tutti).

Un grazie speciale va ad Andrea: ormai non ti vedo più come un semplice collega (tra università e pizzeria… jamm checazz...), e, per quanto ti dispiaccia, mi sono abituato a considerarti un vero amico — e la cosa non può che farmi piacere.

Grazie per avermi accompagnato in questo percorso: dai primi bomboloni post Analisi 1 fino ad oggi ne abbiamo passate tante. Corsi ed esami insieme, lunghi discorsi sulla vita e sui nostri problemi e, come non menzionarli, i "servizi" che hanno colorato molti dei nostri fine giornata. Insomma… che schifo, ma grazie di tutto: senza di te sarebbe stato tutto un po' più monotono.

Mi tocca ringraziare infine qui Lorenzo: grazie, capo. Probabilmente non ti dai il merito, ma grazie alla libertà e alla "buonezza" che trasudano dai tuoi pori, mi hai aiutato a combattere ansie sociali e timidezze varie, presentandomi e lasciandomi in balia di questi scoppiati (di cui ormai mi sento parte a tutti gli effetti).

Mi hai introdotto a loro, e non solo: anche a un gruppo che non ringrazierò mai abbastanza e che, nonostante mi faccia vedere poco (spesso solo nelle grandi occasioni), mi ha sempre fatto sentire benvoluto e mai fuori posto.

Grazie quindi a mamma Laura, Pica, PPP, Marco, Toni, i Kellogs, Vincenzino e tutti voi altri di "Grazia è l'ora dell'anno nuovo" per avermi fatto sentire, ogni tanto, la mascotte del vostro gruppo.

Grazie Ale, non avrei potuto chiedere per un compagno di corso migliore, tra la curiosità e la capacità di reperire qualsiasi tipo di appunto esistente, starei ancora provando qualche esame sicuramente. Non vedo l'ora di condividere qualche punto anche alla magistrale, anche se sai, quanto mi auguro che non facciamo neanche un esame insieme. Grazie per la tua essenza semplice e al contempo profonda.

TONI UE TONI, non potevi mancare mi dispiace, voglio ringraziarti per non aver mai smesso di aver fatto parte della mia, più che quotidianità, diciamo, mensilità, proteggo molto gelosamente il bar o la camminata per fuorigrotta che quasi ogni mesetto ci concediamo, ogni incontro fatto di aggiornamenti e delle tue storie pazze (le mie un po meno lo so) e una boccata d'aria fresca e mi auguro di non interrompere mai questa streak per troppo tempo.
UE TONI

A Facocero (Giuse),

"Questa laurea è anche mia", ripeti spesso da quando mi hai abbandonato (kek) per la pecunia. In effetti non posso darti torto: mi hai tifato e sostenuto da subito, con una sincerità ai miei occhi disarmante, e di questo non mi dimenticherò.

Ammiro la tua socialità e quasi invidio la facilità con cui conosci le persone, ti fai voler bene e rimani in contatto con chiunque incroci. Sappiamo entrambi che spesso mi nascondevo dietro di te mentre facevi amicizia e poi ne approfittavo (kek): esempio Andrea, Caporale, Ale, e potrei nominarne molti altri, ma lascio questo spazio a te.

Nonostante ora tu sia troppo impegnato a fatturare, so che avrai sempre il tempo di ricordarti di mandarmi un selfie mentre sei sul cesso e di urlare "ciao Gianni", e che proveremo sempre a incastrarci, anche solo per una sigaretta giù al palazzo e sentire dei tuoi nuovi piani di vita più disparati possibile.

Grazie, Giuse: anche questa è per te.

Un grazie particolare va a Cirettina che, oltre a "tenermelo buono", si è rivelata una persona di cuore e un'amica su cui poter contare con la genuinità che ti contraddistingue. Grazie.

Al Gazy,

Appena vi ho visti, mi siete sembrati tutti degli scappati di casa e, attenzione, lo dico come un complimento, perché ero esattamente come voi. Non avete esitato un attimo a considerarmi "Massimo Pericolo" mentre, scornuoso come sono, provavo a stare al tavolo con ragazzi che non conoscevo, vi ringrazio quindi per la semplicità con cui mi avete fatto integrare.

Vi ringrazio anche per tutto quello che è venuto dopo: ricordo con nostalgia le tante serate passate in quel gazebino, dove non facevamo altro che ridere, ascoltare musica e quell'altra cosa che, insomma, sappiamo tutti (ma c'era spazio anche per i momenti seri e forse proprio quelli ci hanno aiutato a legarci ancora di più).

Aspetto ancora una rimpatriata come ai vecchi tempi e, se capita, anche un weekend a Ischia non guasterebbe. Anche se siamo cresciuti tutti, vi voglio bene: Vincenzino, Genni, Tonino e Marittiello (gli altri due stessero zitti).

A te invece, bastardo, che non sarai qui… ok, scherzo,

A Bozzarella. A te dovevo dedicare qualche riga in più. Mi dispiace averti "scoperto", per non dire conosciuto, non così in fretta come avrei voluto. Un vicino di casa a dir poco particolare che mi manca parecchio (anche tu mi hai abbandonato per la pecunia).

Dai Bob in videochiamata affacciati al balcone, alle banali passeggiate per andare e tornare dal Gazy, fino alle serate alla Play anche insieme ai ragazzi, con risate che non provo da un po'. Mi fa piacere sapere che ogni volta che "scendi" mi riserverai uno slot serata con Peppone per ridere di nuovo insieme.

Grazie a voi, Mattia, JD, Gabri, Avv e Tatix, per aver contornato ogni momento passato insieme con un mare di risate (sì, anche a te, Gabri).

Un grazie un po' più particolare va a Kekkino che, con la tua spontaneità e curiosità, sei riuscito a regalarmi ancora un'altra prospettiva sul nostro gruppo.

Mi auguro che, se in futuro dovessero esserci altri ringraziamenti, possa solo allungare la lista delle cose che ho da dirvi. Grazie di tutto.

A Genni, Teo, Secchio e Top (ordine random, zitti), la mia seconda famiglia

Con la speranza che queste parole non vengano usate contro di me. Spero di avervi fatti sentire tutti i ringraziamenti prima dei vostri, così, da un lato, vi ho fatto rompere il cazzo, e dall'altro sappiate, come rassicurazione, che anche in vostra assenza (fisica) c'è ancora qualcuno capace di prendersi cura di me.

Su di voi ci sarebbe da scrivere un libro, ma so che vi fermereste a metà della prima pagina (e andreste a cercare solo i vostri nomi), quindi inizio a ringraziarvi insieme: un grazie "generale", ma in realtà il più sentito, per non avermi mai abbandonato e, lo sappiamo, in certi periodi me lo sarei anche meritato.

Con voi ho capito il significato della parola amicizia e quanto, alla fine, il termine "fratellanza" per noi non sia così esagerato, anzi. Approfitto di questa introduzione per ringraziarvi innanzitutto della vostra testardaggine nei miei confronti, di essere cresciuto insieme a voi e di aver condiviso esperienze che porterò per sempre con me.

A Top: ti scrivo per primo sia per continuare il meme del "miglior amico e compagno di banco", sia per il paradosso del fatto che ci conosciamo da quasi tredici anni e sei probabilmente la persona con cui mi sento e mi vedo di meno, e questo non fa altro che farmi ridere. Appena ci vediamo, infatti, bastano due o tre schiaffi a vicenda e torniamo quella coppia di compagni di banco disadattati alla Carlo Poerio.

Ti scrivo queste righe perché mi piacerebbe sottolineare che, quando scherzi dicendo "grazie per i compiti" e altre stronzate (come il 96…), sono io che devo ringraziare te per il bene che mi vuoi, per aver sempre preso le mie parti e avermi scelto, sempre, in tutti questi anni scolastici.

Un grazie particolare va alla tua famiglia, che mi ha sfamato e accolto sin da subito.

Grazie, Andriu.

A Secchio,

#TiOdio e non sarai mai il mio Leader. Lo dico perché, sotto quelle tante "maschere" che alla fine, esteriormente, un po' ti contraddistinguono — i vari Raspadori, Messi, Hamsik e così via — so che c'è un vero ragazzo d'oro, che a volte preferisce mostrarsi un pelo più superficialmente.

Lo dico avendo conosciuto, o almeno così mi piace pensare, il vero Ale, che preferisco di gran lunga rispetto al Leader, per il suo animo molto più puro di quanto un esterno possa immaginare, e ti sono grato per avermelo fatto conoscere. Ti stimo più di quanto credi e molte volte sei stato tu ad aiutarmi, e a calmarmi, anche se non sempre te ne ho dato il merito.

Sii sempre te stesso e non aver paura di mostrarti per ciò che sei, che tutto andrà come deve andare. Capisc a me.

A Teo,

se dovessi reinventare il termine "inclusione", probabilmente avrebbe il tuo nome. Non mi hai mai fatto sentire escluso da nulla e, per quanti gruppi potessero esserci senza di me, non è mai mancato un tuo messaggio per avvisarmi di un'uscita o di qualsiasi altra cosa. Approfitto per chiederti scusa: a volte non ti ho trattato bene come avrei dovuto e non avrei mai dovuto sfogare dello di stress su di te, che hai sempre speso energie per avermi con voi, dal "non andartene così presto" alle uscite prima degli esami e tante altre occasioni.

So che in ogni frase che adesso usiamo per sfotterci c'è tanto bene, che ricambio sinceramente, e mi auguro di poterlo fare da qui a quando non ti trasferirai in Svizzera o in Portogallo (gagagagagaga).

Grazie, Teo, per avermi dato la possibilità di conoscerti oltre quel rozzo barbaro che provi a essere quando stiamo insieme, e grazie per il tuo valore come amico, per come credi nel gruppo e nella compagnia.

Una menzione speciale a Giadina, ti ringrazio per mantenere ancora umano Teo, e nonostante non abbiamo avuto molte occasioni per stringere veramente, per darmi quella

sicurezza che appena ne avremo l'occasione sarai una grande amica oltre al vedere le tue unghie.

A Genni,

non scambierei la nostra amicizia per nulla al mondo. Sei tra le persone su cui più posso contare e, nell'ultimo periodo, ne ho avuto l'ennesima dimostrazione. Anche tu hai un mondo dentro, difficilmente accessibile: oltre al ragazzo tatuato, muscoloso e con fare imprenditoriale, ho felicemente scoperto che c'è molto altro.

La tua lealtà, riservata a pochi, la considero un privilegio. Il bene che provi verso di noi è qualcosa di speciale, soprattutto per chi è in grado di vederlo, magari non avendo paura di sfidarti (cosa che a me invece diverte molto).

Mi hai insegnato tante cose, tra tutte, il non arrendersi mai, il guardare un problema come un evento inevitabile per arrivare a qualcosa di meglio dopo. Porterò con me la tua fame e la consapevolezza di avere un fratello felice per i miei traguardi, tanto quanto lo sarò io per i tuoi.

Mangiati il mondo e, quando vuoi, mi porti ad assaggiarlo. Grazie, GenniGetto.